

Értékmentő

Digitális játékonysági platform Vizsgaremek Dokumentáció

Készítette:

Kemény Benedek

Papp Bence

Budapest, 2026

Tartalomjegyzék

1. Bevezetés.....	8
1.1 A téma ismertetése és a témaválasztás indoklása.....	8
1.2 A program célja és célközönség	8
1.3 Hasonló alkalmazások és az Értékmentő különlegessége.....	9
1.4 Köszönetnyilvánítás	10
1.5 Elérhetőségek	10
2. Fejlesztői dokumentáció	11
2.1 Fejlesztői környezet	11
2.1.1 Hardverkörnyezet	11
2.1.2 Fejlesztőeszközök és szoftverek	12
2.1.3 Alkalmazott programnyelvek és keretrendszerek	12
2.1.4 A technológiai döntések indoklása	13
2.2 Az alkalmazás architektúrája	14
2.2.1 A rétegek közötti adatáramlás	15
2.2.2 Generic Repository és Unit of Work pattern	15
2.3 Adatszerkezetek – az adatbázis részletes bemutatása.....	15
2.3.1 Counties – Vármegyék	15
2.3.2 ProductCategory – Termék kategóriák	16
2.3.3 Users – Felhasználók	17
2.3.4 Products – Termékek	18
2.3.5 ProductRequests – Termékigénylések	19
2.3.6 RequesterLimitRule – Igénylési limitszabályok.....	20
2.3.7 RequesterLimitUsage – Igénylési limit-felhasználás.....	21
2.3.8 Chat – Csevegőszálak.....	22
2.3.9 ChatMessage – Chat üzenetek	23
2.4 Fontosabb algoritmusok.....	24

2.4.1 Regisztrációs folyamat.....	24
2.4.2 Bejelentkezés és JWT token generálás	25
2.4.3 Termék feltöltése képpel	25
2.4.4 Termékigénylési folyamat és limit-ellenőrzés	26
2.4.5 Chat üzenetküldés és valós idejű kommunikáció	27
2.4.6 Olvasatlan üzenetek számlálása.....	27
2.5 Tesztdokumentáció	28
2.5.1 Tesztkörnyezet.....	28
2.5.2 Tesztfelhasználók	28
2.5.3 Funkcionális tesztek.....	29
2.5.4 Határesetek és hibakezelés tesztelése	30
2.5.5 API-tesztelés Swagger segítségével.....	35
2.6 Fejlesztési lehetőségek	42
3. Felhasználói dokumentáció.....	43
3.1 A program célja és főbb funkciói.....	43
3.2 A program elindításához szükséges eszközök és szoftverek	44
3.2.1 Szükséges hardvereszközök	44
3.2.2 Szükséges szoftverek	44
3.3 Az alkalmazás elérése.....	45
3.4 A főoldal bemutatása	46
3.4.1 Hero szekció	46
3.4.2 Küldetésünk szekció	47
3.4.3 Eredmények szekció	48
3.4.4 Gyakran Ismételt Kérdések (GYIK)	49
3.4.5 Kapcsolati szekció.....	50
3.5 Regisztráció	51
3.5.1 Bejelentkezés	52

3.5.2 Igénylő felhasználó termékek böngészése felület	53
3.5.3 Igénylő igénylései.....	54
3.5.4 Igénylő Felhasználó Chat felülete	55
3.5.5 Felajánló termékei.....	56
3.5.6 Felajánló chat felülete	57
3.5.7 Felajánló igénylés lezárása	58
3.5.8 Admin felhasználó felhasználók kezelése	59
3.5.9 Admin felhasználó: termékek kezelésének felülete	60
3.5.10 Admin felhasználó: igénylések kezelésének felülete.....	61
3.5.11 Admin felhasználó: limitek kezelésének felülete	62
3.5.12 A regisztrációs mezők leírása és elvárások.....	62
3.5.13 Lehetséges hibaüzenetek regisztráció során.....	63
3.6 Bejelentkezés.....	64
3.6.1 A bejelentkezés lépései.....	64
3.6.2 Bejelentkezési hibaüzenetek	64
3.7 Termékek böngészése (Igénylői nézet)	65
3.7.1 Szűrési lehetőségek	65
3.7.2 Termékek megjelenítése	65
3.7.3 Termék részleteinek megtekintése	66
3.8 Termék felajánlása (Adományozói nézet)	66
3.8.1 Új felajánlás létrehozásának lépései	66
3.8.2 A termék-feltöltési mezők leírása.....	66
3.9 Igénylések kezelése.....	67
3.9.1 Igénylői nézet – Saját igényléseim	67
3.9.2 Adományozói nézet – Beérkezett igénylések	67
3.10 Chat – Üzenetváltás.....	68
3.11 Adminisztrátori felület.....	68

3.11.1 Felhasználókezelés (Admin → Felhasználók)	68
3.11.2 Termékkezelés (Admin → Termékek).....	68
3.11.3 Igényléskezelés (Admin → Igénylések)	69
3.11.4 Limitszabály-kezelés (Admin → Limitszabályok)	69
3.12 Kijelentkezés	69
3.13 Regisztráció nélkül elérhető funkciók	70
3.14 Elérhetőség, információkérés	70
4. Összefoglalás.....	71
4.1 Az elkészült munka értékelése	71
4.2 Tanulságok és személyes fejlődés	71
4.3 A program jövőbeli hasznosítása.....	72
4.4 Köszönetnyilvánítás.....	72
5. Irodalomjegyzék	73
5.1 Online dokumentációk és webes forrásanyagok	73
5.2 Szakmai könyvek.....	73
6. Melléklet – DTO-k és interfészek részletes leírása	74
6.1 RegisterDto – Regisztrációs adatátviteli objektum.....	74
6.2 LoginDto és LoginResponseDto	75
6.3 ProductDto – Termékadat-átviteli objektum.....	76
6.4 ProductRequestDto – Igénylés adatátviteli objektum	77
6.5 ChatMessageRequestDto és ChatMessageResponseDto	78
6.6 RequesterLimitRuleDto – Limitszabály adatátviteli objektum	79
7. Melléklet – Az API végpontok teljes dokumentációja	80
7.1 AuthController – Hitelesítési végpontok.....	80
7.2 ProductController – Termékkezelési végpontok	80
7.3 ProductRequestController – Igényléskezelési végpontok.....	81
7.4 ChatController – Chat végpontok	83

7.5 LimitController – Limit-kezelési végpontok.....	83
7.6 AdminController – Adminisztrátori végpontok	84
7.7 LookupController – Referenciaadat-lekérések	85
8. Melléklet – Az adatbázis-séma teljes SQL-szkriptje és kapcsolatok.....	86
8.1 Táblák közötti kapcsolatok	86
8.2 Adatbázis indexek és azok indoklása.....	88
8.3 Az adatbázis-séma teljes létrehozó szkriptjének részletes magyarázata	89
9. Melléklet – A Blazor WEB réteg részletes bemutatása.....	91
9.1 Az alkalmazás oldalainak listája.....	91
9.2 Hitelesítési architektúra a Blazor kliensen.....	92
9.3 Az oldalszintű jogosultság-ellenőrzés	93
9.4 A SignalR valós idejű chat implementáció	93
10. Melléklet – Konfigurációs fájlok és üzembe helyezés	95
10.1 Az appsettings.json konfiguráció.....	95
10.2 Program.cs – A middleware pipeline konfigurációja	96
10.3 Fejlesztői környezet felállítása (Getting Started)	97
11. Melléklet – Részletes tesztesetek.....	98
11.1 Regisztrációs tesztek	98
11.2 Bejelentkezési tesztek.....	99
11.3 Termékkezelési tesztek.....	100
11.4 Igénylési tesztek.....	101
11.5 Chat tesztek	102
11.6 Adminisztrátori tesztek	103
11.7 Reszponzivitás és böngésző-kompatibilitás tesztek.....	104
12. Melléklet – A projekt teljes könyvtárszerkezete	106
13. Melléklet – Enum értékek és státuszgépek.....	108
13.1 ProductStatus – Termék állapotgép	108

13.2 RequestStatus – Igénylés állapotgép	109
13.3 UserRole – Felhasználói szerepkörök	109
13.4 UserStatus – Felhasználói státuszok.....	110
14. Melléklet – Biztonsági szempontok és megoldások	111
14.1 Jelszóbiztonság – BCrypt hash-elés.....	111
14.2 JWT token biztonság	111
14.3 CORS konfiguráció.....	112
14.4 SQL Injection védelem – EF Core ORM.....	112

1. Bevezetés

1.1 A téma ismertetése és a témaválasztás indoklása

Az Értékmentő egy digitális jótekonysági platform, amelynek célja, hogy közvetlen kapcsolatot teremtsen azok között, akik feleslegessé vált, de még jó állapotú tárgyaikat szívesen felajánlanák, és azok között, akiknek ezekre valóban szükségük van. A mindennapok során gyakran előfordul, hogy a háztartásokban felhalmozódnak különböző tárgyak – például bútorok, elektronikai eszközök, ruhák vagy könyvek –, amelyeket már nem használunk, mégsem szeretnénk kidobni őket. Ezzel párhuzamosan viszont sok olyan ember él, akinek ezek a dolgok nagy segítséget jelentenének, de anyagi okokból nem tudják megvásárolni őket.

A témaválasztás részben személyes tapasztalaton alapul. Saját élmény volt megtapasztalni, hogy a jelenlegi, főként apróhirdetési rendszerek mennyire nehézkesen működnek ilyen célra. Ezeken a felületeken az adományozási szándék gyakran nem találkozik a valódi igényekkel, hiszen nincs külön kezelve a jótekonysági felajánlás és az értékesítés. Emellett hiányoznak olyan fontos funkciók is, mint például a területi szűrés vagy az igénylések korlátozása, így akár az is előfordulhat, hogy egyetlen felhasználó több felajánlást is lefoglal, mások elől elvéve a lehetőséget.

Az Értékmentő erre készült. A platform kizárólag jótekonysági célú: minden felajánlás ingyenes, a felhasználók külön szerepkörrel rendelkeznek (adományozó vagy igénylő), és kategóriánként szabályozható, ki mennyi terméket igényelhet egy adott időszakban. Az ország teljes területét lefedi, így a felhasználók a lakóhelyükhöz közeli felajánlásokat is megtalálják.

1.2 A program célja és célközönség

Az Értékmentő platform célja kettős. Egyrészt segíteni kívánja az adományozókat abban, hogy felesleges tárgyaik könnyen, gyorsan és célzottan eljuthassanak olyan emberekhez, akiknek valóban szükségük van rájuk – nem kell azokat hirdetési díjakért megjelentetni, nem kell alkudozni, és az átadás menetéről a felek a beépített chat-funkcióval egyeztetnek. Másrészt a platform

a rászorulóknak számára biztosít egy átlátható, kategóriák és vármegyék szerint szűrhető felületet, ahol kényelmesen megkereshetik, amire szükségük van, és egyetlen gombnyomással igényt nyújthatnak be.

Az Értékmentő két oldalt szolgál ki. Az adományozóknak egyszerű felületet ad: feltöltik a tárgyat, az igénylővel a beépített chaten egyeztetnek, és kész. Nem kell hirdetési díjat fizetni, nem kell alkudozni. Az igénylők kategória és vármegye szerint szűrve böngészhetik a felajánlásokat, és egy gombbal jelezhetik igényüket.

Három szerepkör létezik: Adományozó, Igénylő és Admin. Az adminisztrátorok kezelik a felhasználókat, moderálják a tartalmakat, és állítják be az igénylési limiteket.

1.3 Hasonló alkalmazások és az Értékmentő különlegessége

A játékonysági tárgycsere területén már léteznek megoldások, amelyek részben hasonló célt szolgálnak. A Hasznaltcikk.hu és az Apróhirdetés.hu oldalak például lehetővé teszik ingyenes tárgyak hirdetését, de ezek általános célú piacterek, ahol az ingyenes és fizetős ajánlatok keverednek, nincs dedikált játékonysági szűrő, és nincsenek szerepköralapú korlátok. A Facebook Marketplace szintén lehetővé tesz ingyenes adományozást csoportokon belül, de a csoportok szétaprózottság jellemzi, és nem biztosítanak semmilyen igénylési szabályozást.

Az Értékmentő ezektől az alábbi szempontokban tér el. Kizárólag játékonysági jellegű: nem lehet termékeket eladni, csak ingyenesen felajánlani.

Szerepköralapú rendszert alkalmaz, amelyben az adományozók és az igénylők külön kezelve vannak. Adminisztrátori igénylési limitszabályok biztosítják, hogy egy felhasználó ne foglalhassa le az összes tárgyat. Az integrált chat-rendszer lehetővé teszi az egyeztetést az átadás részleteiről, anélkül, hogy a felek elhagynák az alkalmazást. Vármegye alapú szűrés teszi lehetővé a regionális böngészést. Mindezek miatt az Értékmentő nem csupán egy hirdetési oldal, hanem egy célzott játékonysági közösségi eszköz.

1.4 Köszönetnyilvánítás

A fejlesztők köszönetüket fejezik ki felkészítő tanáraiknak, tesztelőiknek és családjaiknak. A részletes köszönetnyilvánítás a 4.4-es fejezetben olvasható.

1.5 Elérhetőségek

Az alkalmazással, a dokumentációval vagy bármely szakmai kérdéssel kapcsolatban az alábbi e-mail-elérhetőségeken lehet felvenni a kapcsolatot:

- Kemény Benedek – kemeny.benedek.mate@ganziskola.hu
- Papp Bence – papp.bence@ganziskola.hu

2. Fejlesztői dokumentáció

A fejlesztői dokumentáció az alkalmazás technikai megvalósítását mutatja be részletesen. Célja, hogy egy szakmabeli fejlesztő a dokumentáció alapján megértse az Értékmentő belső felépítését, az alkalmazott technológiákat és architektúrát, az adatbázis szerkezetét, a főbb algoritmusokat, valamint a tesztelési szempontokat.

2.1 Fejlesztői környezet

2.1.1 Hardverkörnyezet

Az alkalmazás fejlesztése a következő hardverkörnyezetben zajlott:

Komponens	Specifikáció
Fejlesztői gép	Windows 10/11 operációs rendszer, Intel Core i5/i7 processzor, 16 GB RAM, SSD tárhely
Adatbázis-szerver	Helyi SQL Server Express futtatásra alkalmas gép (BENIPC\SQLEXPRESS példány)
Webszerver (fejlesztői)	ASP.NET Core Kestrel belső webszerver (localhost:5000 / HTTPS: 7xxx)
Böngésző	Google Chrome, Microsoft Edge (legfrissebb verzió)
Operációs rendszer	Windows 11

2.1.2 Fejlesztőeszközök és szoftverek

A fejlesztés során az alábbi szoftverek és eszközök kerültek felhasználásra:

Eszköz / Szoftver	Verzió / Megjegyzés	Szerepe a projektben
Visual Studio 2022	Community Edition	Fő IDE, C# kódfejlesztés, Blazor komponensek
SQL Server Express	2022	Relációs adatbázis-kezelő rendszer
SQL Server Management Studio (SSMS)	19.x	Adatbázis kezelése, lekérdezések futtatása
Git	2.x	Verziókezelés
GitHub	Felhő alapú	Kóddal együttműködés, repository tárolása
Swagger UI	OpenAPI 3.0	Automatikus API dokumentáció és tesztelés
dbdesigner.net	Online eszköz	Adatbázis-diagram tervezése

2.1.3 Alkalmazott programnyelvek és keretrendszerek

Technológia	Verzió	Felhasználási terület
C#	.NET 8	Backend logika, API, Blazor szerver oldal
ASP.NET Core Web API	.NET 8	REST API réteg

Blazor Server	.NET 8	Szerver oldali webes felhasználói felület
Entity Framework Core	8.x	ORM – adatbázis-kezelés C#-ból
MS SQL Server / T-SQL	2021+	Relációs adatbázis és SQL szkriptek
BCrypt.Net	4.x	Jelszó hash-elés
JWT (JSON Web Token)	Bearer Auth	Felhasználó-hitelesítés tokenek alapján
SignalR	ASP.NET Core	Valós idejű chat kommunikáció
HTML5 / CSS3	Szabvány	Blazor komponensek stílusozása
Bootstrap 5	5.x	Reszponzív UI keretrendszer

2.1.4 A technológiai döntések indoklása

A C# és az ASP.NET Core választása részben praktikus döntés volt – a szakmai vizsgán is ehhez kapcsolódó feladatok várhatók, így a projekt egyben vizsgafelkészülés is volt. Stabil környezetet akartunk, és a .NET 8 ezt hozta.

A Blazor Server mellett azért döntöttünk, mert így mindketten C#-ban dolgozhattunk végig – frontenden és backenden egyaránt. A másik reális opció a Next.js lett volna, de azt egyikőnk se ismerte rendesen, és nem akartunk a fejlesztés közepén tanulni egy új keretrendszert. Az MS SQL Server és az Entity Framework Core az adatbázis-kezelést fedte le, a JWT a hitelesítést, a SignalR pedig a valós idejű chatet.

2.2 Az alkalmazás architektúrája

Az Értékmentő Clean Architecture szerint épül fel, ami azt jelenti, hogy az alkalmazás öt egymástól elkülönített projektre van bontva. A lényege: ha az adatbázis-réteget kell módosítani, a frontend-kódot nem kell hozzányúlítani, és fordítva.

Réteg / Projekt	Felelőssége	Főbb fájlok
Domain	Üzleti entitások, enum-ok, tartománymodellek. Semmitől sem függ.	User.cs, Product.cs, Chat.cs, ProductRequest.cs, Enum-ok
Application	Üzleti logika, service interfészek, DTO-k, service implementációk. Csak a Domain-től függ.	IAuthService.cs, AuthService.cs, ProductDto.cs, LoginDto.cs stb.
Infrastructure	Adatbázis-hozzáférés, repository-k, JWT generálás, EF Core DbContext, SignalR Hub. Az Application-től függ.	ErtekmentoDbContext.cs, GenericRepository.cs, JwtTokenGenerator.cs, ChatHub.cs
API	REST API végpontok (kontrollerek), Swagger konfiguráció, CORS, autentikáció middleware.	Program.cs, AuthController.cs, ProductController.cs, ChatController.cs stb.
WEB	Blazor Server frontend, oldalkomponensek, navigáció, LocalStorage-kezelés.	Home.razor, Products.razor, Chat.razor, Login.razor stb.

2.2.1 A rétegek közötti adatáramlás

Az alkalmazásban az adatok az alábbi úton áramlanak a felhasználói felülettől az adatbázisig és vissza. A Blazor komponens HTTP-kérést küld az API rétegnek (ApiService-eken keresztül, HttpClient segítségével). A kontroller fogadja a kérést, és átadja a service-nek. Az Application réteg service-e elvégzi az üzleti logikát (validálás, limit-ellenőrzés, stb.), majd az Infrastructure réteg repository-jain keresztül kommunikál az adatbázissal. A Unit of Work pattern gondoskodik a tranzakciók összefogásáról. Az eredmény visszafelé haladva DTO objektumok formájában kerül a klienshez.

2.2.2 Generic Repository és Unit of Work pattern

A projekt a Generic Repository mintát alkalmazza, amelynek köszönhetően az összes entitásra azonos alapműveleteket (GetAllAsync, GetByIdAsync, AddAsync, Update, Delete) lehet alkalmazni egyetlen általános implementáción keresztül. Az IGenericRepository<T> interfész típusparaméterrel definiálja ezeket a metódusokat.

A Unit of Work (IUnitOfWork) összefogja az összes repository-t, és a CompleteAsync() metóduson keresztül egy lépésben menti el az összes változást az adatbázisba. Így a service rétegnek nem kell közvetlenül a DbContext-tel dolgoznia.

2.3 Adatszerkezetek – az adatbázis részletes bemutatása

Az alkalmazás MS SQL Server adatbázist használ, amelynek neve Ertekmento_db. Az adatbázis összesen 10 táblából áll, amelyek a játékonysági platform összes adatát tárolják. Az alábbiakban minden tábla részletes leírása következik, a mezők nevével, típusával, hosszával és szerepével.

2.3.1 Counties – Vármegyék

A Counties tábla Magyarország 20 vármegyét tárolja. Erre azért van szükség, mert a termékeket és az igényléseket vármegyénként is szűrni lehet. A tábla tartalmát az alkalmazás indulásakor az adatbázis-szkript tölti fel, és nem módosítható a futó alkalmazáson keresztül.

Mező neve	Típus	Tulajdonságok	Leírás
CountyId	INT	PK, IDENTITY(1,1)	Egyedi azonosító, automatikusan növekszik
CountyName	NVARCHAR(100)	NOT NULL, UNIQUE	A vármegye neve (pl. Budapest, Pest, Győr-Moson-Sopron)

Feltöltött értékek: Bács-Kiskun, Baranya, Békés, Borsod-Abaúj-Zemplén, Budapest, Csongrád-Csanád, Fejér, Győr-Moson-Sopron, Hajdú-Bihar, Heves, Jász-Nagykun-Szolnok, Komárom-Esztergom, Nógrád, Pest, Somogy, Szabolcs-Szatmár-Bereg, Tolna, Vas, Veszprém, Zala.

2.3.2 ProductCategory – Termék kategóriák

A ProductCategory tábla a felajánlható termékek kategóriáit tartalmazza.

Minden terméknek pontosan egy kategóriája van. Az IsActive mező lehetővé teszi, hogy egyes kategóriákat adminisztrátori beavatkozás nélkül ideiglenesen letiltsunk anélkül, hogy a rekordot törölnénk.

Mező neve	Típus	Tulajdonságok	Leírás
ProductCategoryId	INT	PK, IDENTITY(1,1)	Egyedi azonosító
ProductCategoryName	NVARCHAR(100)	NOT NULL, UNIQUE	Kategória neve (pl. Bútor, Elektronika, Ruházkodás)
IsActive	BIT	NOT NULL, DEFAULT 1	Aktív-e a kategória (1 = igen, 0 = nem)

Feltöltött értékek: Élelmiszer, Ruházkodás, Bútor, Elektronika, Könyv & Oktatás, Gyógyszer & Egészség, Játék & Hobbi, Egyéb.

2.3.3 Users – Felhasználók

A Users tábla az összes regisztrált felhasználót tárolja, beleértve az adományozókat, az igénylőket és az adminisztrátorokat. Az egyes szerepköröket a UserRole integer mező különíti el. A jelszó BCrypt algoritmussal hashelt formában tárolódik, soha nem plaintext formában.

Mező neve	Típus	Tulajdonságok	Leírás
UserId	INT	PK, IDENTITY(1,1)	Egyedi felhasználói azonosító
Email	NVARCHAR(100)	NOT NULL, UNIQUE	A felhasználó e-mail-címe
PasswordHash	NVARCHAR(255)	NOT NULL	BCrypt hash-elt jelszó (mindig 60 karakter)
UserName	NVARCHAR(50)	NOT NULL, UNIQUE	Felhasználónév (egyedi)
UserRole	INT	NOT NULL	1 = Sender (adományozó), 2 = Requester (igénylő), 3 = Admin
FirstName	NVARCHAR(100)	NOT NULL	Keresztnév
LastName	NVARCHAR(100)	NOT NULL	Vezetéknév

UserStatus	INT	NOT NULL, DEFAULT 1	1=Aktív, 2=Törölve, 3=Felfüggesztett, 4=Tiltott
CreatedAt	DATETIME2	NOT NULL, DEFAULT GETUTCDATE()	Regisztráció időpontja (UTC)
UpdatedAt	DATETIME2	NOT NULL, DEFAULT GETUTCDATE()	Utolsó módosítás időpontja

2.3.4 Products – Termékek

A Products tábla az összes felajánlott adományterméket tárolja. Minden termékhez tartozik egy adományozó (Sender), egy kategória és egy vármegye. A ProductStatus enum alapú állapotgép szabályozza a termék életciklusát.

Mező neve	Típus	Tulajdonságok	Leírás
ProductId	INT	PK, IDENTITY(1,1)	Egyedi termékazonosító
SenderId	INT	NOT NULL, FK → Users	A felajánló felhasználó azonosítója
ProductCategoryId	INT	NOT NULL, FK → ProductCategory	A termék kategóriájának azonosítója
CountyId	INT	NOT NULL, FK → Counties	Melyik vármegyében érhető el a termék

ProductName	NVARCHAR(100)	NOT NULL	A termék neve (pl. Asztal, Mosógép)
ProductDescription	NVARCHAR(MAX)	NULL	A termék leírása, állapota, részletek
ImagePath	NVARCHAR(500)	NULL	A feltöltött kép szerver-oldali relatív útvonala
ProductStatus	INT	NOT NULL	1=Aktív, 2=Függőben, 3=Átadva, 4=Törölve
CreatedAt	DATETIME2	NOT NULL, DEFAULT GETUTCDATE()	Feltöltés időpontja
UpdatedAt	DATETIME2	NOT NULL, DEFAULT GETUTCDATE()	Utolsó módosítás időpontja

Indexek: IX_Products_CategoryId (gyors kategória-szűrés), IX_Products_SenderId (adományozó saját termékei), IX_Products_CountyId (vármegye szerinti szűrés).

2.3.5 ProductRequests – Termékigénylések

A ProductRequests tábla rögzíti, mikor és ki igényelt egy adott terméket. Egy termékre egyszerre csak egy aktív (Pending státuszú) igénylés lehet. Az igénylés lezárásakor (Completed vagy Failed) a termék státusza is frissítésre kerül.

Mező neve	Típus	Tulajdonságok	Leírás
ProductRequestId	INT	PK, IDENTITY(1,1)	Egyedi igénylés-azonosító
ProductId	INT	NOT NULL, FK → Products (CASCADE DELETE)	Az igényelt termék azonosítója
RequesterId	INT	NOT NULL, FK → Users	Az igénylő felhasználó azonosítója
RequestStatus	INT	NOT NULL	1=Függőben, 2=Teljesítve, 3=Sikertelen
RequestedAt	DATETIME2	NOT NULL, DEFAULT GETUTCDATE()	Igénylés benyújtásának időpontja
ProcessedAt	DATETIME2	NULL	Igénylés feldolgozásának időpontja (ha lezárták)

2.3.6 RequesterLimitRule – Igénylési limitszabályok

Mező neve	Típus	Tulajdonságok	Leírás
RequesterLimitRuleId	INT	PK, IDENTITY(1,1)	Egyedi szabályazonosító
RequesterLimitRuleCategoryId	INT	NOT NULL, FK → ProductCategory	Melyik kategóriára vonatkozik a szabály

PeriodType	NVAR CHAR(50)	NOT NULL, CHECK IN (Weekly, Monthly, Quarterly, Semiannual)	Az időszak típusa	A
MaxQuantity	INT	NOT NULL, CHECK > 0	Maximum igényelhető mennyiség az adott időszakban	
RequesterLimit RuleDescription	NVAR CHAR(500)	NULL	Opcionális leírás a szabályhoz	
IsActive	BIT	NOT NULL, DEFAULT 1	Aktív-e a szabály	
CreatedAt	DATETIME2	DEFAULT GETUTCDATE()	Létrehozás időpontja	
UpdatedAt	DATETIME2	DEFAULT GETUTCDATE()	Módosítás időpontja	

RequesterLimitRule tábla kategóriánként definiálja, hogy egy igénylő adott időszakon belül (heti, havi, negyedéves, féléves) maximum hány terméket igényelhet. Ez a mechanizmus megakadályozza, hogy egyetlen felhasználó az összes felajánlott terméket lefoglalhassa.

2.3.7 RequesterLimitUsage – Igénylési limit-felhasználás

A RequesterLimitUsage tábla nyomon követi, hogy az egyes igénylők az egyes szabályok alapján az aktuális időszakban hány terméket igényeltek már. A UNIQUE constraint biztosítja, hogy felhasználónként, szabályonként és időszak-kezdetenként csak egy rekord létezzen, elkerülve az adatduplikációt.

Mező neve	Típus	Tulajdonságok	Leírás
RequesterLimitUsageld	INT	PK, IDENTITY(1,1)	Egyedi rekordazonosító
RequesterId	INT	NOT NULL, FK → Users	Az igénylő felhasználó azonosítója
RuleId	INT	NOT NULL, FK → RequesterLimitRule	A vonatkozó limitszabály azonosítója
PeriodStart	DATETIME2	NOT NULL	Az aktuális időszak kezdetének dátuma
PeriodEnd	DATETIME2	NOT NULL	Az aktuális időszak végének dátuma
UsedQuantity	INT	NOT NULL, DEFAULT 0	Az időszokban eddig felhasznált igénylések száma
LastResetAt	DATETIME2	NOT NULL, DEFAULT GETUTCDATE()	Utolsó reset időpontja

UNIQUE constraint: UC_RequesterLimitUsage (RequesterId, RuleId, PeriodStart) – garantálja, hogy egy felhasználóra, egy szabályra és egy időszak-kezdetre csak egy sor létezhet.

2.3.8 Chat – Csevegőszálak

A Chat tábla egy-egy igényléshez tartozó csevegőszálat (szobát) tartalmaz. Minden igényléshez pontosan egy chat tartozhat (UNIQUE a ProductRequestId mezőn). A chat az adományozó és az igénylő közötti kommunikációt biztosítja.

Mező neve	Típus	Tulajdonságok	Leírás
ChatId	INT	PK, IDENTITY(1,1)	Egyedi chat-azonosító
ProductRequestId	INT	NOT NULL, UNIQUE, FK → ProductRequests (CASCADE)	A kapcsolódó igénylés azonosítója
SenderId	INT	NOT NULL, FK → Users	Az adományozó felhasználó azonosítója
RequesterId	INT	NOT NULL, FK → Users	Az igénylő felhasználó azonosítója
CreatedAt	DATETIME2	NOT NULL, DEFAULT GETUTCDATE()	A chat létrehozásának időpontja

2.3.9 ChatMessage – Chat üzenetek

A ChatMessage tábla az egyes chat-szálakhoz tartozó üzeneteket tartalmazza. Az IsRead mező lehetővé teszi az olvasatlan üzenetek nyomon követését, a ReadAt mező pedig rögzíti az olvasás időpontját. A SenderId mező alapján különíthető el, hogy melyik fél küldte az üzenetet.

Mező neve	Típus	Tulajdonságok	Leírás
ChatMessageId	INT	PK, IDENTITY(1,1)	Egyedi üzenetazonosító
ChatId	INT	NOT NULL, FK → Chat (CASCADE)	A szülő chat azonosítója

SenderId	INT	NOT NULL, FK → Users	Az üzenetet küldő felhasználó azonosítója
ChatMessage	NVARCHAR(MAX)	NOT NULL	Az üzenet szöveges tartalma
IsRead	BIT	NOT NULL, DEFAULT 0	Olvasta-e már a másik fél (0=nem, 1=igen)
SentAt	DATETIME2	NOT NULL, DEFAULT GETUTCDATE()	Küldés időpontja
ReadAt	DATETIME2	NULL	Olvasás időpontja (ha már olvasva)

Index: IX_ChatMessage_ChatId – a chat-szálhoz tartozó üzenetek gyors lekérdezéséhez.

2.4 Fontosabb algoritmusok

2.4.1 Regisztrációs folyamat

A regisztráció az AuthController /api/Auth/register végpontján keresztül indul el. A bejövő RegisterDto objektum tartalmazza a felhasználó nevét, e-mail-címét, felhasználónevét, jelszavát és a szerepkör azonosítóját.

Az AuthService.RegisterAsync() metódusban az alábbi lépések hajtódnak végre:

1. Az összes meglévő felhasználó lekérése az adatbázisból.
2. Ellenőrzés, hogy létezik-e már azonos e-mail-cím vagy felhasználónév. Ha igen, a metódus false-szal tér vissza, és a kontroller 400 Bad Request választ küld.
3. Új User objektum létrehozása, amelybe a jelszó BCrypt.Net.BCrypt.HashPassword() metódussal hashelt formában kerül. A BCrypt alapból 11 salt-round-ot alkalmaz, ami elegendő biztonsági szintet nyújt.
4. Az új felhasználó hozzáadása az adatbázishoz a Unit of Work pattern CompleteAsync() metódusán keresztül.

5. Sikeres mentés esetén true-val való visszatérés, amit a kontroller 200 OK válaszként jelez a kliensnek.

```
var newUser = new User {  
    UserName = registerDto.UserName,  
    Email = registerDto.Email,  
    PasswordHash =  
BCrypt.Net.BCrypt.HashPassword(registerDto.Password),  
    UserRole = (Domain.Enums.UserRole)registerDto.RoleId,  
    UserStatus = UserStatus.Active  
};
```

A Blazor WEB rétegben a regisztrációs form `DataAnnotationsValidator` segítségével kliens oldalon is érvényesíti a beviteli mezőket, ezzel minimalizálva a felesleges szerverkéréseket.

2.4.2 Bejelentkezés és JWT token generálás

A bejelentkezés a `POST /api/Auth/login` végponton valósul meg. A folyamat lépései:

6. Az `AuthService.LoginAsync()` metódus az összes felhasználót lekéri, majd az e-mail-cím vagy felhasználónév alapján megkeresi a megfelelő rekordot.
7. A `BCrypt.Net.BCrypt.Verify()` metódussal összehasonlítja a megadott jelszót a tárolt hash-sel. Ha nem egyeznek, a metódus null-lal tér vissza.
8. A kontroller ellenőrzi, hogy a user státusza Active-e. Inaktív, tiltott vagy törölt fiók esetén 401 Unauthorized választ küld.
9. Sikeres azonosítás esetén a `JwtTokenGenerator.GenerateToken()` metódus HS256 algoritmussal aláírt JWT tokenet állít elő.

A token a következő claim-eket tartalmazza: felhasználói azonosító (`Nameldentifier`), felhasználónév (`Name`), keresztnév (`GivenName`), vezetéknév (`Surname`), szerepkör (`Role`) és e-mail-cím (`Email`). A token 2 óráig érvényes. A Blazor kliensen a token a `LocalStorage`-ban kerül tárolásra, és minden API-híváshoz az `Authorization: Bearer {token}` fejlécbe illesztve kerül elküldésre.

2.4.3 Termék feltöltése képpel

A termék létrehozása a `POST /api/Product` végponton valósul meg, `multipart/form-data` formátumban, hogy a kép is átvihető legyen. A `ProductService.CreateProductAsync()` metódus logikája:

10. Ha a feltöltött fájl (IFormFile) nem null és mérete nagyobb nullánál, a rendszer GUID-alapú egyedi fájlnevet generál (pl. 3f8a2b1c-...jpg).
11. A kép a wwwroot/images/products/ mappába kerül mentésre. Ha a mappa nem létezik, a rendszer automatikusan létrehozza.
12. A kép szerver-oldali elérési útvonala (/images/products/{uuid}.jpg) a ProductDto.ImagePath mezőjébe kerül.
13. A Product entitás létrehozásra és mentésre kerül az adatbázisban.

```
var fileName =  
$" {Guid.NewGuid()} {Path.GetExtension(imageFile.FileName)} ";  
var filePath = Path.Combine(uploadsFolder, fileName);  
using (var stream = new FileStream(filePath, FileMode.Create)) {  
    await imageFile.CopyToAsync(stream);  
}
```

2.4.4 Termékigénylési folyamat és limit-ellenőrzés

A termékigénylés a POST /api/ProductRequest/claim/{productId} végponton kezdődik. A ProductRequestService.ClaimProductAsync() metódus az alábbi üzleti logikát valósítja meg:

14. Ellenőrzi, hogy a termék Active státuszú-e. Nem aktív terméket nem lehet igényelni.
15. Ellenőrzi, hogy van-e már aktív (Pending státuszú) igénylés a termékre. Ha igen, az igénylés visszautasításra kerül.
16. Meghívja a LimitService.CanUserRequestProduct() metódust, amely megállapítja, hogy a felhasználónak van-e még szabad kapacitása az adott kategóriában.
17. Ha minden feltétel teljesül, létrehozza a ProductRequest rekordot Pending státusszal.
18. Frissíti a termék státuszát Active-ről Pending-re.
19. Meghívja a LimitService.UpdateLimitUsage() metódust, amely növeli a felhasználó limit-felhasználási számát az adott időszakra.

A LimitService.GetPeriod() segédmetódusa a limitszabályban definiált időszak típus alapján kiszámítja az aktuális periódus kezdetét és végét:

```
"weekly" => hét első napja (vasárnap 00:00) - vasárnap 23:59  
"monthly" => hónap első napja - következő hónap első napja
```

2.4.5 Chat üzenetküldés és valós idejű kommunikáció

A chat-rendszer két részből áll: egy REST API rétegből (üzenetek küldése és előzmények lekérése) és egy SignalR hubból (valós idejű értesítések). Az üzenetküldés folyamata a `ChatService.SendMessageAsync()` metódusban:

20. Az igénylés azonosítója alapján a metódus megkeresi a vonatkozó `ProductRequest` rekordot.
21. A termék azonosítóján keresztül lekéri a terméket és megállapítja az adományozó azonosítóját.
22. Ellenőrzi, hogy létezik-e már chat a megadott igénylés-azonosítóhoz. Ha nem, új Chat rekordot hoz létre.
23. Új `ChatMessage` rekordot hoz létre a küldő azonosítójával, az üzenet szövegével és az aktuális UTC időbélyeggel.
24. A `ChatHub` `SignalR` hub a csatlakozó klienseket `requestId` alapján csoportokba rendezi (`JoinChat` metódus). Üzenetküldéskor a szerver az adott csoportba tartozó összes kliensnek elküldi az értesítést, így a másik fél azonnal látja az üzenetet oldal-frissítés nélkül.

2.4.6 Olvasatlan üzenetek számlálása

Az alkalmazás fejlécén megjelenő olvasatlan üzenetek száma az alábbi algoritmussal kerül kiszámításra:

25. Az összes aktív (Pending státuszú) igénylés azonosítójának összegyűjtése egy `HashSet<int>`-be.
26. Azon chat-szálak azonosítóinak összegyűjtése, amelyekben a bejelentkezett felhasználó résztvevő (`SenderId` vagy `RequesterId`), és az igénylés aktív.
27. A `ChatMessage` táblából azon üzenetek megszámlálása, amelyek a felhasználó chat-szálaihoz tartoznak, `IsRead = false` értékűek, és küldőjük nem a bejelentkezett felhasználó.

2.5 Tesztdokumentáció

2.5.1 Tesztkörnyezet

Az alkalmazás tesztelése az alábbi környezetekben zajlott:

Tesztkörnyezet	Operációs rendszer	Böngésző	Felbontás
Asztali számítógép (fejlesztői)	Windows 11 x64	Google Chrome 123	1920×1080
Asztali számítógép	Windows 10 x64	Mozilla Firefox 124	1920×1080
Laptop	Windows 11 x64	Microsoft Edge 123	1366×768
Táblagép (szimuláció)	Ipad OS	iPad nézet	768×1024
Okostelefon (szimuláció)	IOS	iPhone 16 nézet	390×844

2.5.2 Tesztfelhasználók

Az alábbi tesztfelhasználók állnak rendelkezésre a fejlesztői teszteléshez (regisztráció nélkül):

E-mail / Felhasználónév	Jelszó	Szerepkör	Hozzáférési szint
Felado / userSender@teszt.com	123123	Sender (Adományozó)	Termékek feltöltése, szerkesztése, törlése; chat
Igenylo/ userRequester@teszt.com	123123	Requester (Igénylő)	Termékek böngészése, igénylés, chat

Admin / useradmin@teszt.com	123123	Admin	Teljes adminisztrátori hozzáférés
--------------------------------	--------	-------	-----------------------------------

2.5.3 Funkcionális tesztek

Az alábbi táblázat összefoglalja a fontosabb funkcionális tesztek eredményeit:

Teszt eset azonosító	Leírás	Bemenet	Várt eredmény	Eredmény
TC-001	Regisztráció érvényes adatokkal	Új, egyedi e-mail és felhasználónév, érvényes jelszó	200 OK, felhasználó létrehozva	PASS
TC-002	Regisztráció már létező e-maillal	Meglévő e-mail-cím	400 Bad Request	PASS
TC-003	Bejelentkezés helyes adatokkal	Érvényes e-mail és jelszó	200 OK + JWT token	PASS
TC-004	Bejelentkezés helytelen jelszóval	Helyes e-mail, hibás jelszó	401 Unauthorized	PASS
TC-005	Termék feltöltése képpel	Termék adatok + JPG kép	200 OK, kép megjelenik	PASS
TC-006	Termék igénylése (limit alatt)	Érvényes productId, Requester szerepkör	200 OK, igénylés létrehozva	PASS

TC-007	Termék igénylése (limit elérve)	Limit-tel ellátott kategória, már elért limit	400 Bad Request + üzenet	PASS
TC-008	Már igényelt termék igénylése	Pending státuszú termék	400 Bad Request	PASS
TC-009	Chat üzenet küldése	Érvényes requestId, üzenettartalom	200 OK, üzenet megjelenik	PASS
TC-010	Admin: felhasználó tiltása	UserId, Admin szerepkör	200 OK, UserStatus=Banned	PASS

2.5.4 Határesetek és hibakezelés tesztelése

A rendszer robusztusságának ellenőrzésére az alábbi határeseteket teszteltük:

A rendszer hibakezelési képességének ellenőrzése érdekében az alábbi hibás bemenetekkel és határesetekkel is elvégeztük a tesztelést. Minden hibás esetben a rendszer megfelelő HTTP státuszkóddal és érthető hibaüzenettel reagált.

Hibás regisztráció: már létező e-mail-cím vagy felhasználónév megadásakor a rendszer 400 Bad Request választ küld, és jelzi a duplikációt. Az alábbi képernyőképen látható, ahogy a Swagger UI megjeleníti a hibás regisztrációra adott választ.

The screenshot displays the Swagger UI interface. At the top, a cURL command is shown for a POST request to `https://localhost:7161/api/Auth/register`. The request body is a JSON object with fields: `userName`, `email`, `password`, `confirmPassword`, `firstName`, `lastName`, and `roleId`. Below the cURL command, the 'Request URL' is `https://localhost:7161/api/Auth/register`. The 'Server response' section shows a 400 status code. The 'Response body' is a JSON object indicating an error: `{ "type": "https://tools.ietf.org/html/rfc9110#section-15.5.1", "title": "One or more validation errors occurred.", "status": 400, "errors": { "UserName": [ "A felhasználónév kötelező" ] }, "traceId": "00-efe9855799063275c87edc5fbf34c562-79de23b50e7a6b2f-00" }`. The 'Response headers' are: `content-type: application/problem+json; charset=utf-8`, `date: Thu, 09 Apr 2026 13:07:08 GMT`, and `server: Kestrel`. The 'Responses' table at the bottom shows a 400 status code with the description 'Error: response status is 400'.

1. kép: Hibás regisztráció – a rendszer 400 Bad Request választ küld, ha az e-mail-cím már foglalt

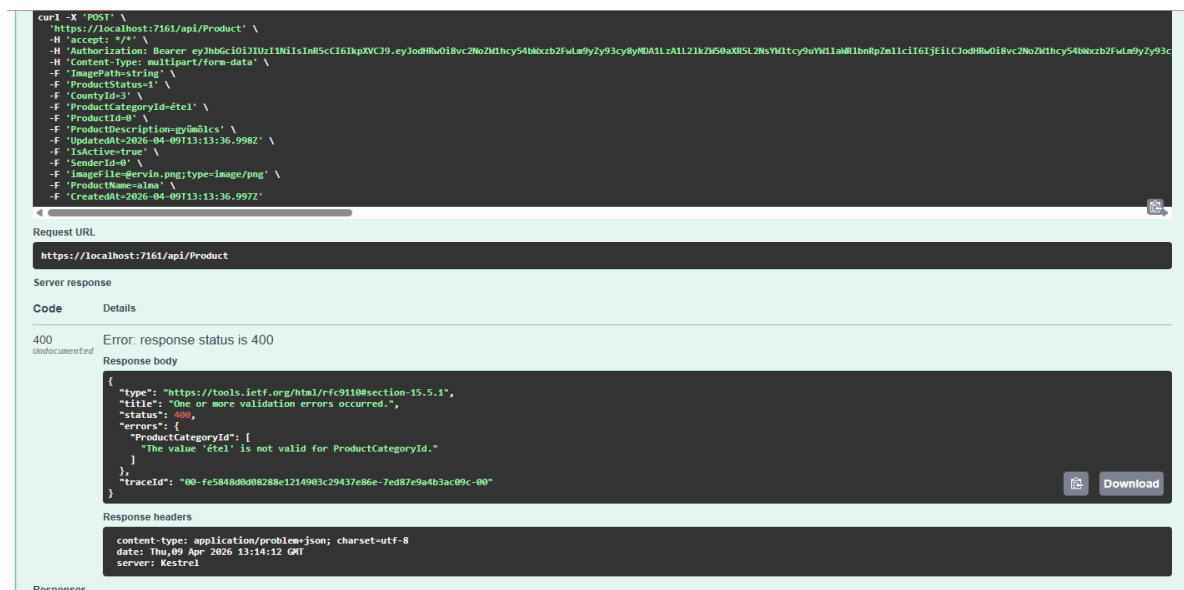
Sikertelen bejelentkezés: helytelen jelszó megadásakor a végpont 401

Unauthorized státuszkóddal tér vissza, és nem kerül token generálásra. A következő képen megfigyelhető a sikertelen bejelentkezési kísérlet Swagger UI-on.

The screenshot displays the Swagger UI interface. At the top, a cURL command is shown for a POST request to `https://localhost:7161/api/Auth/login`. The request body is a JSON object with fields: `emailOrUserName` and `password`. Below the cURL command, the 'Request URL' is `https://localhost:7161/api/Auth/login`. The 'Server response' section shows a 401 status code. The 'Response body' is a JSON object indicating an error: `{ "type": "https://tools.ietf.org/html/rfc9110#section-15.5.2", "title": "Unauthorized", "status": 401, "traceId": "00-5ecfb955a830d19649ff14556a59c535-8769867feaff4868-00" }`. The 'Response headers' are: `content-type: application/problem+json; charset=utf-8`, `date: Thu, 09 Apr 2026 13:09:58 GMT`, and `server: Kestrel`. The 'Responses' table at the bottom shows a 401 status code with the description 'Error: response status is 401'.

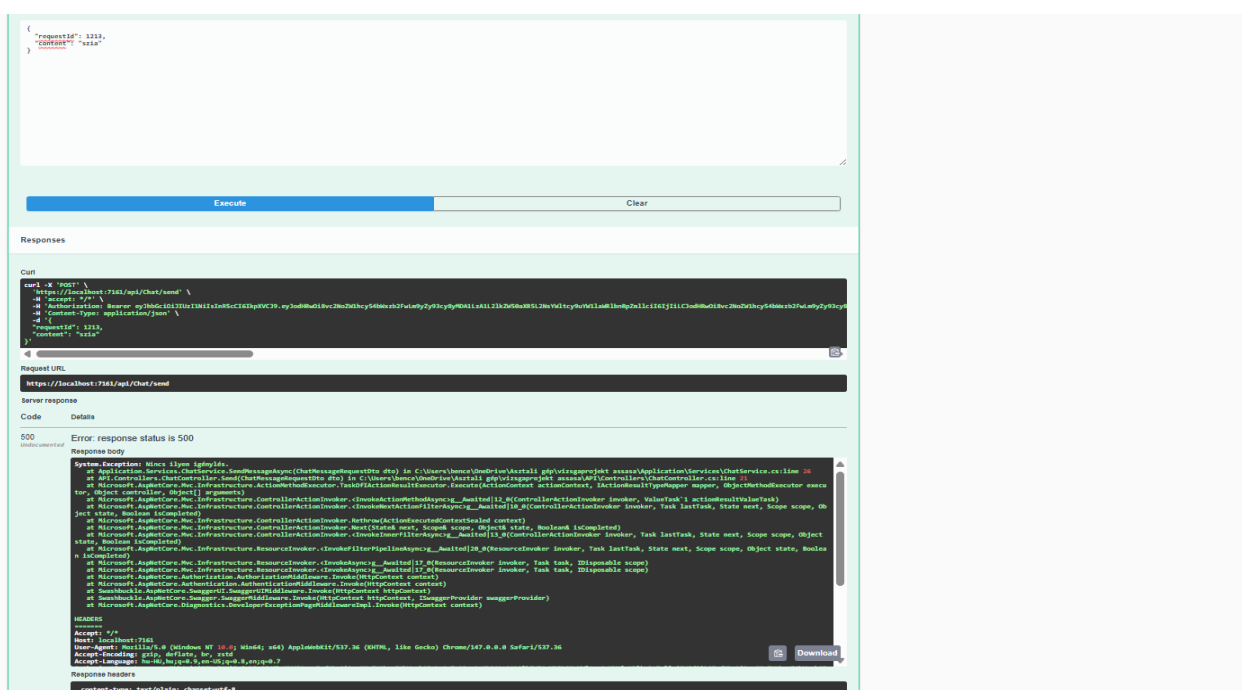
2. kép: Sikertelen bejelentkezés – hibás jelszó esetén 401 Unauthorized válasz érkezik

Hibás termékfeltöltés: kötelező mezők elhagyása esetén a szerver validációs hibát jelez vissza. Az alábbi képernyőkép azt mutatja, hogyan reagál a rendszer hiányos termékadatok beküldésekor.



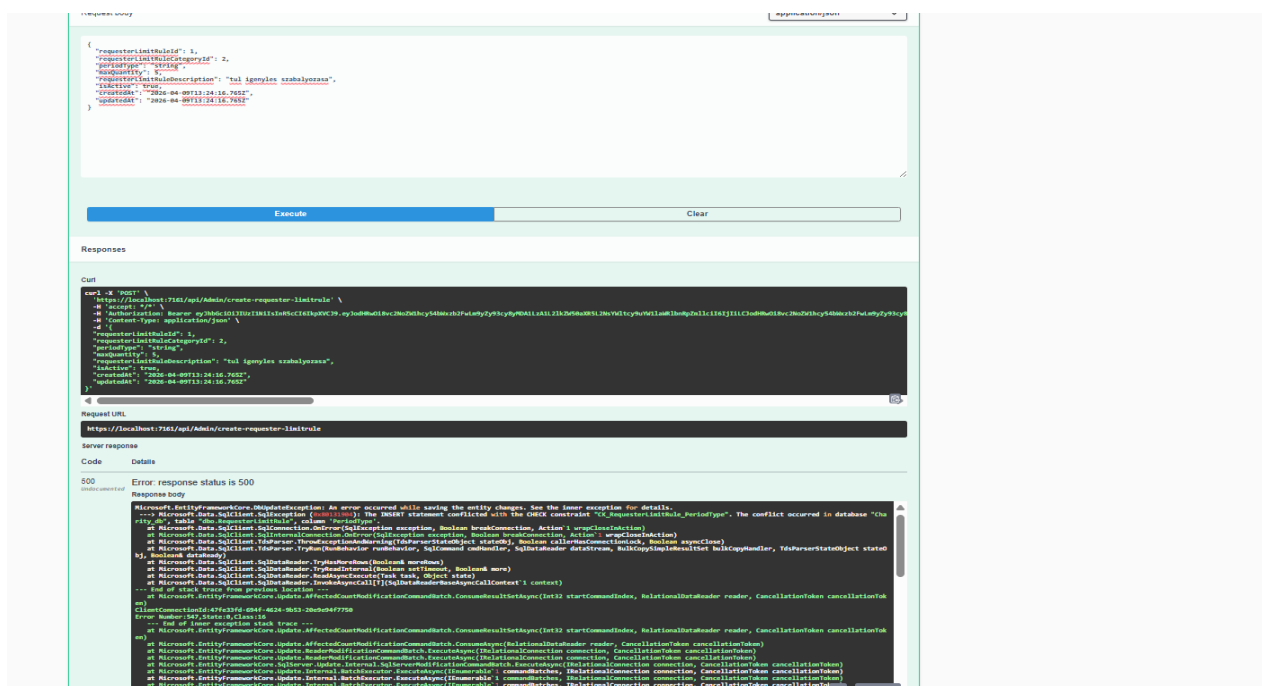
3. kép: Hibás termékfeltöltés – hiányos adatok esetén 400 Bad Request státusz kód jelenik meg

Hibás üzenetküldés: érvénytelen igénylés-azonosítóval küldött üzenet esetén a rendszer 404 Not Found vagy 400 Bad Request választ ad vissza. A következő képen látható a hibás üzenetküldési kísérlet eredménye.



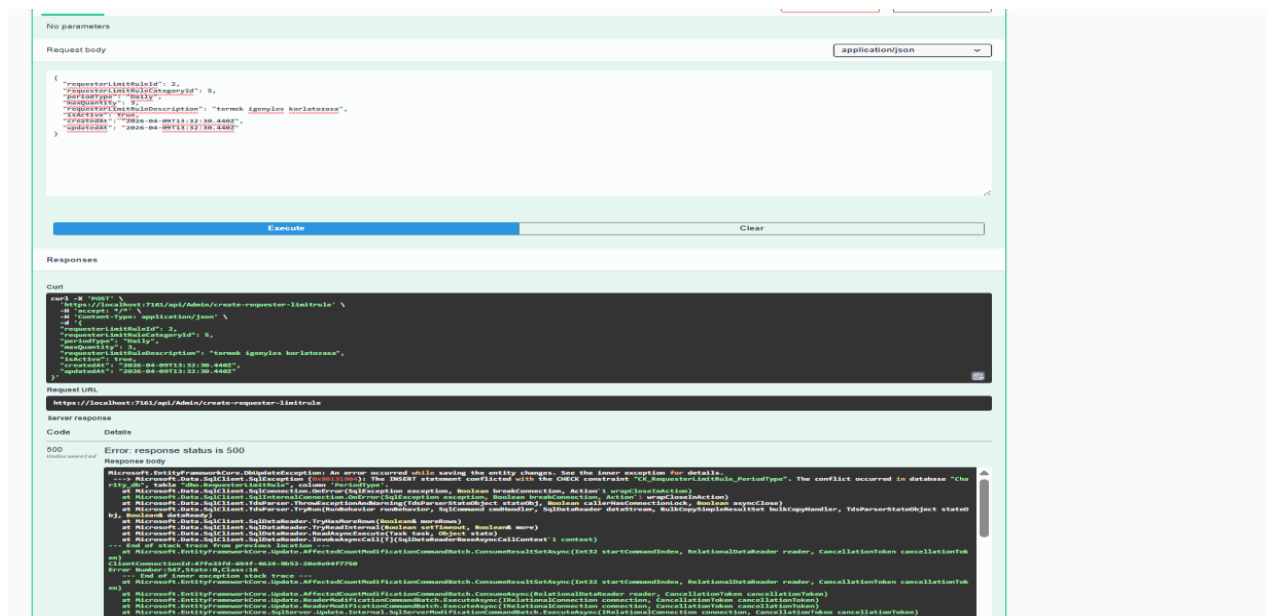
4. kép: Hibás üzenetküldés – érvénytelen azonosító esetén a rendszer hibaüzenettel reagál

Hibás limitszabály-létrehozás: érvénytelen időszak-típus vagy nulla alatti mennyiség megadásakor a rendszer visszautasítja a kérést. Az alábbi képen megfigyelhető a hibás limitszabály-létrehozási kísérlet.



5. kép: Hibás limitszabály-létrehozás – érvénytelen paraméterek esetén a rendszer 400 Bad Request státuszt küld

Jogosulatlan admin végpont hívás: nem adminisztrátori jogosultságú tokennel hívott admin végpont esetén a rendszer 403 Forbidden vagy 401 Unauthorized választ ad. A következő képen látható egy ilyen jogosulatlan hozzáférési kísérlet.



6. kép: Jogosulatlan admin végpont hívás – nem megfelelő jogosultság esetén a rendszer megtagadja a hozzáférést

2.5.5 API-tesztelés Swagger segítségével

Az API-végpontok tesztelése a Swagger UI felületen keresztül történt, amely az ASP.NET Core Swashbuckle csomag által automatikusan generált interaktív API dokumentáció. A Swagger felület a fejlesztői (Development) futtatási környezetben a `https://localhost:{port}/swagger` URL-en érhető el.

A tesztelés lépései minden védett végpontnál azonosak voltak: először a POST `/api/Auth/login` végponton keresztül bejelentkeztünk egy tesztfelhasználóval, majd a visszakapott JWT tokenet az Authorize gombra kattintva, Bearer {token} formátumban adtuk meg. Ezt követően az egyes végpontok a lakat ikonnal jelzik az aktív hitelesítési állapotot.

Az alábbi képen a sikeres regisztrációs kísérlet látható Swagger UI-on. A POST `/api/Auth/register` végpont 200 OK státuszkóddal tért vissza, jelezve, hogy az új felhasználó sikeresen létrejött.

The screenshot displays the Swagger UI interface for a REST API. It shows a curl command for a POST request to `https://localhost:7161/api/Auth/register` with a JSON body containing user registration details. The response is a 200 OK status with headers indicating a successful registration.

```
Curl
curl -X 'POST' \
  'https://localhost:7161/api/Auth/register' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "userName": "au",
    "email": "peledepelelda.com",
    "password": "123123",
    "confirmPassword": "123123",
    "firstName": "Kis",
    "lastName": "Hiklós",
    "roleId": 1
  }'
```

Request URL

`https://localhost:7161/api/Auth/register`

Server response

Code	Details
200	Response headers <pre>content-length: 0 date: Thu, 09 Apr 2026 13:05:10 GMT server: Kestrel</pre>

Responses

Code	Description	Links
------	-------------	-------

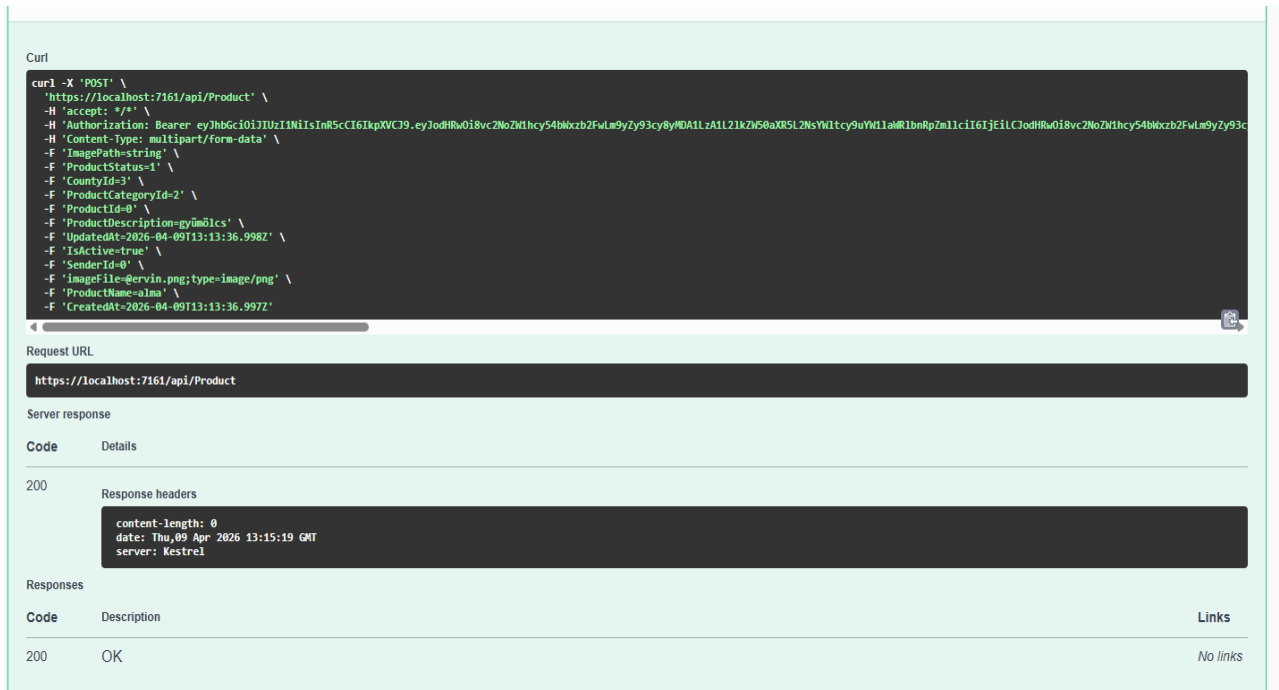
7. kép: Sikeres regisztráció – a Swagger UI 200 OK választ jelenít meg az új felhasználó létrehozásakor

A sikeres bejelentkezési tesztnél a POST /api/Auth/login végpont 200 OK válasszal és JWT tokennel tért vissza. A következő képen megfigyelhető a Swagger felületen megjelenő sikeres bejelentkezési válasz.

[illegible]

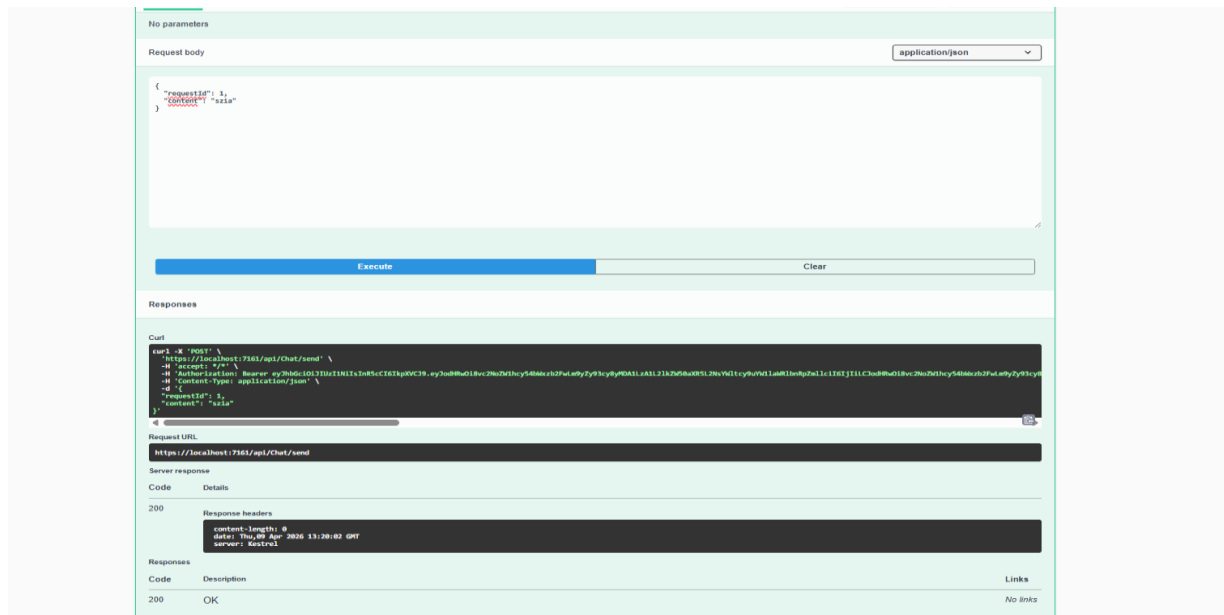
8. kép: Sikeres bejelentkezés – 200 OK válasz és JWT token a Swagger UI-on

A termékfeltöltési teszt során a POST /api/Product végpontra multipart/form-data formátumban küldtük az adatokat. A sikeres feltöltés 200 OK választ adott vissza. Az alábbi képernyőkép a sikeres termékfeltöltés eredményét mutatja.



9. kép: Sikeres termékfeltöltés – a Swagger UI 200 OK státuszt és a termék adatait jeleníti meg

Az üzenetküldési tesztnél a POST /api/Chat/send végpontot hívtuk meg érvényes requestId és üzenettartalommal. A következő képen látható a sikeres üzenetküldés Swagger UI-on.



10. kép: Sikeres üzenetküldés – 200 OK válasz az üzenet elküldése után a Swagger UI-on

A limitellenőrzési tesztnél a limit alatt lévő kategóriában igényeltük a terméket. A rendszer engedélyezte az igénylést és frissítette a limit-felhasználási számlálót. Az alábbi képen látható a sikeres limitellenőrzés eredménye.

[illegible]

11. kép: Sikeres limitellenőrzés – a rendszer engedélyezi az igénylést, ha a limit még nem merült ki

A limitszabály sikeres létrehozásakor a POST /api/Limit/rule végpont 200 OK választ ad vissza az újonnan létrehozott szabály adataival együtt. A következő képen megfigyelhető az új limitszabály létrehozásának eredménye.

The screenshot displays a REST client interface with the following sections:

- Request body:** A JSON object defining a limit rule with fields like `requesterLimitRuleId`, `requesterLimitRuleCategoryId`, `periodType`, `maxQuantity`, `requesterLimitRuleDescription`, `isActive`, `createdAt`, and `updatedAt`.
- Execute:** A button to send the request.
- Responses:** A section showing the result of the request.
 - Curl:** A terminal window showing the equivalent curl command.
 - Request URL:** `https://localhost:7161/api/Admin/create-requester-limitrule`
 - Server response:** A table showing a `200` status code.

Code	Details
200	Response headers <code>content-length: 0</code> <code>date: Thu, 04 Apr 2026 13:27:02 GMT</code> <code>server: Kestrel</code>
 - Responses:** A table summarizing the response.

Code	Description	Links
200	OK	No links

12. kép: Sikeres limitszabály-létrehozás – az admin által létrehozott új szabály 200 OK válasszal visszaigazolásra kerül

Az adminisztrátori felhasználó tiltási teszténél a PUT /api/Admin/users/{id}/ban végpontot hívtuk meg admin tokennel. A végpont 200 OK választ adott, és a felhasználó státusza Banned értékre változott. Az alábbi képen látható a sikeres tiltási művelet.

[illegible]

13. kép: Felhasználó tiltása admin jogosultsággal – a rendszer 200 OK választ ad és a státusz Banned-re vált

A tesztelés során összesen 13 végpontra vonatkozó sikeres és hibás tesztet futtatására került sor. A Swagger felület a 200, 400, 401 és 403 HTTP státuszkódokat helyesen jeleníti meg, és a hibakeresés során az ezekből levonható következtetések segítették a fejlesztési hibák azonosítását és javítását.

2.6 Fejlesztési lehetőségek

Az Értékmentő jelenlegi verziója egy működőképes MVP (Minimum Viable Product). A jövőbeli fejlesztési irányok:

E-mail értesítések: Az igénylések állapotváltozásakor (elfogadva, elutasítva, lezárva) automatikus e-mail értesítés küldése az érintett felhasználóknak SMTP integráció segítségével (pl. SendGrid, Mailjet).

Értékelési rendszer: Az átadás befejezése után mindkét fél értékelhesse a másik felhasználót, elősegítve a megbízható közösség kialakulását.

Mobilalkalmazás: React Native alapú keresztplatformos mobilalkalmazás fejlesztése, amely az API-val kommunikál, és push-értesítéseket küld.

Admin-dashboard statisztikák: Interaktív grafikonok az adományozási trendek, legnépszerűbb kategóriák és aktív régiók megjelenítéséhez.

Képtimalizálás: Feltöltéskor automatikus képméretezés és WebP konverzió SixLabors.ImageSharp könyvtárral, csökkentve a szerver terhelését és a betöltési időt.

Kétfaktoros azonosítás (2FA): TOTP alapú kétfaktoros hitelesítés integrálása a fiók biztonsági szintjének növelésére.

3. Felhasználói dokumentáció

A felhasználói dokumentáció az Értékmentő platformot azoknak szól, akik nem szoftverfejlesztők, de szeretnék igénybe venni az alkalmazás szolgáltatásait. Ez a fejezet feltételezi, hogy az olvasó semmit sem tud az alkalmazásról, ezért az alapoktól kezdve, lépésről lépésre ismerteti a használatát.

3.1 A program célja és főbb funkciói

Az Értékmentő egy ingyenes, webalapú jótékonyági platform, amelyen magánszemélyek számukra már felesleges, de jó állapotú tárgyaikat felajánlhatják azoknak, akiknek szükségük van rájuk. A platform használatához semmilyen pénzt nem kell fizetni – sem az adományozó, sem az igénylő részéről.

Az alkalmazás főbb funkciói az alábbiak:

- Termékek böngészése: Az igénylők kategória és vármegye szerint szűrve böngészhetik a felajánlott tárgyakat.
- Termék felajánlása: Az adományozók feltölthetik felesleges tárgyaikat fotóval, leírással és helyszín megjelölésével.
- Igénylés benyújtása: Az igénylők egy gombnyomással igényelhetnek egy terméket.
- Chat-kommunikáció: Az adományozó és az igénylő valós időben kommunikálhat az átadás részleteiről az integrált chat-rendszeren keresztül.
- Igénylési korlátozás: A rendszer megakadályozza, hogy egyetlen felhasználó aránytalanul sok terméket igényeljen egy adott időszakban.
- Adminisztrátori felület: Az adminisztrátorok kezelhetik a felhasználókat, termékeket, igényléseket és limitszabályokat.

Fontos tudni: bizonyos funkciók csak regisztrált és bejelentkezett felhasználók számára elérhetők. Regisztráció nélkül kizárólag a nyitóoldal tájékoztató

tartalmai olvashatók, a termékek böngészése és az igénylések leadása csak bejelentkezés után lehetséges.

3.2 A program elindításához szükséges eszközök és szoftverek

3.2.1 Szükséges hardvereszközök

Az Értékmentő egy webalkalmazás, amelynek használatához internet-hozzáférésre alkalmas eszközre van szükség. Az alábbiakban az egyes eszköztípusokhoz tartozó minimális rendszerkövetelmények találhatók:

Eszköztípus	Minimális konfiguráció
Asztali számítógép / Laptop	Windows 7+, macOS 10.12+, vagy Linux; legalább 1 GB RAM, internetkapcsolat
Táblagép	Android 5.0+, iOS 12+; legalább 1 GB RAM, internetkapcsolat
Okostelefon	Android 5.0+, iOS 12+; legalább 1 GB RAM, internetkapcsolat

3.2.2 Szükséges szoftverek

Az alkalmazás használatához kizárólag egy naprakész webböngészőre van szükség. Külön szoftver telepítése nem szükséges. Az alábbi böngészők mindegyike teljes mértékben támogatott:

Böngésző	Minimális verzió	Elérhetőség
Google Chrome	90+	https://www.google.com/chrome/
Mozilla Firefox	88+	https://www.mozilla.org/firefox/

Microsoft Edge	90+	https://www.microsoft.com/edge/
Apple Safari	14+	macOS és iOS beépített böngésző
Opera	76+	https://www.opera.com/

3.3 Az alkalmazás elérése

Az Értékmentő webalkalmazás egy böngészőn keresztül érhető el. A hozzáférés lépései:

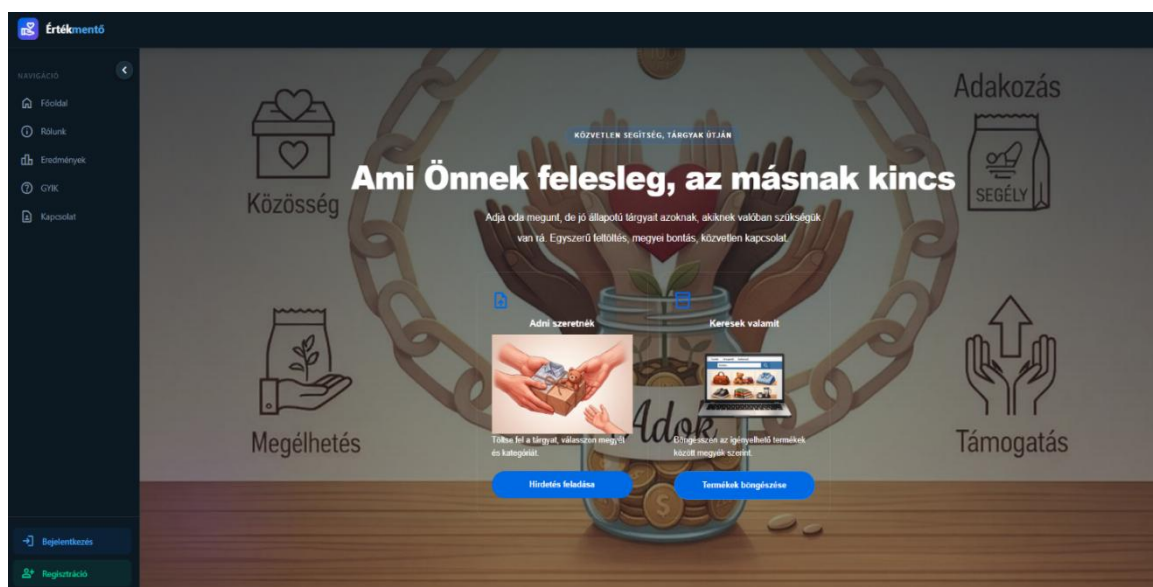
28. Győződjünk meg arról, hogy az eszközünk csatlakoztatva van az internethez.
29. Nyissuk meg a választott webböngészőt (pl. Google Chrome, Firefox).
30. A böngésző felső részén található keresési/cím sávba írjuk be az alkalmazás webcímét, vagy kattintsunk a megfelelő linkre.
31. Üssük le az Enter billentyűt. Az alkalmazás főoldala megjelenik.

Ha az alkalmazás az intézményi szerveren fut, a pontos URL-t az intézmény határozza meg. Fejlesztői (teszt) környezetben az alkalmazás a [www.https://ertek-mento.hu](https://ertek-mento.hu) címen érhető el.

3.4 A főoldal bemutatása

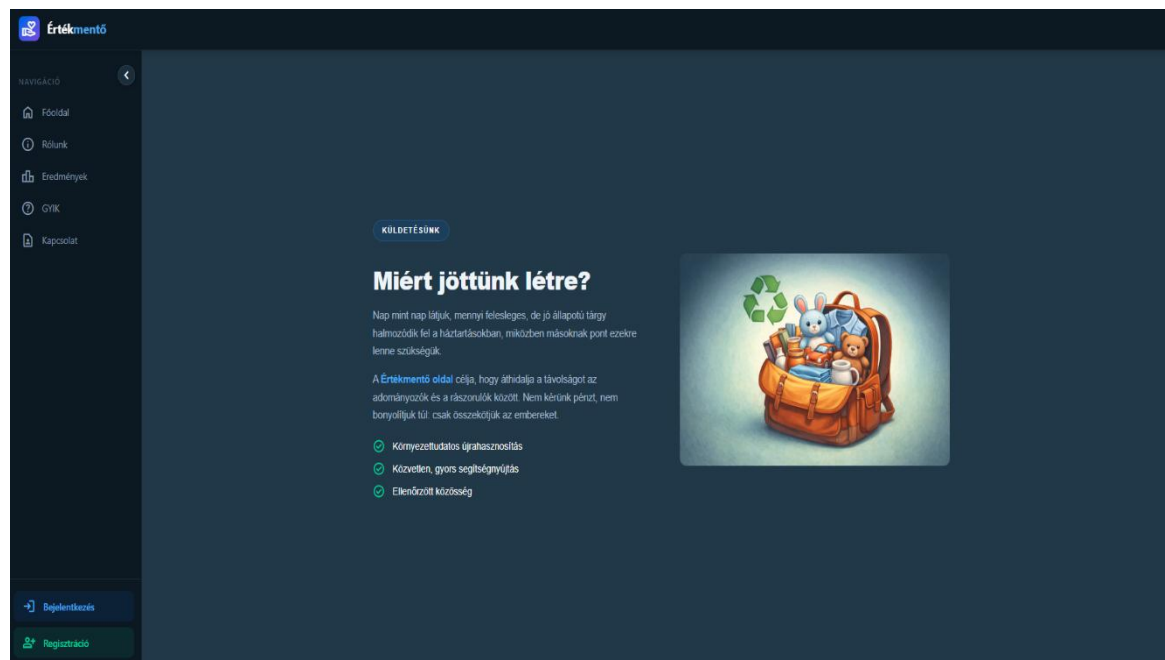
Az alkalmazás megnyitásakor a főoldal fogadja a látogatót. A főoldal az alábbi fő szekciókat tartalmazza, amelyek között lefelé görgetéssel lehet navigálni:

3.4.1 Hero szekció



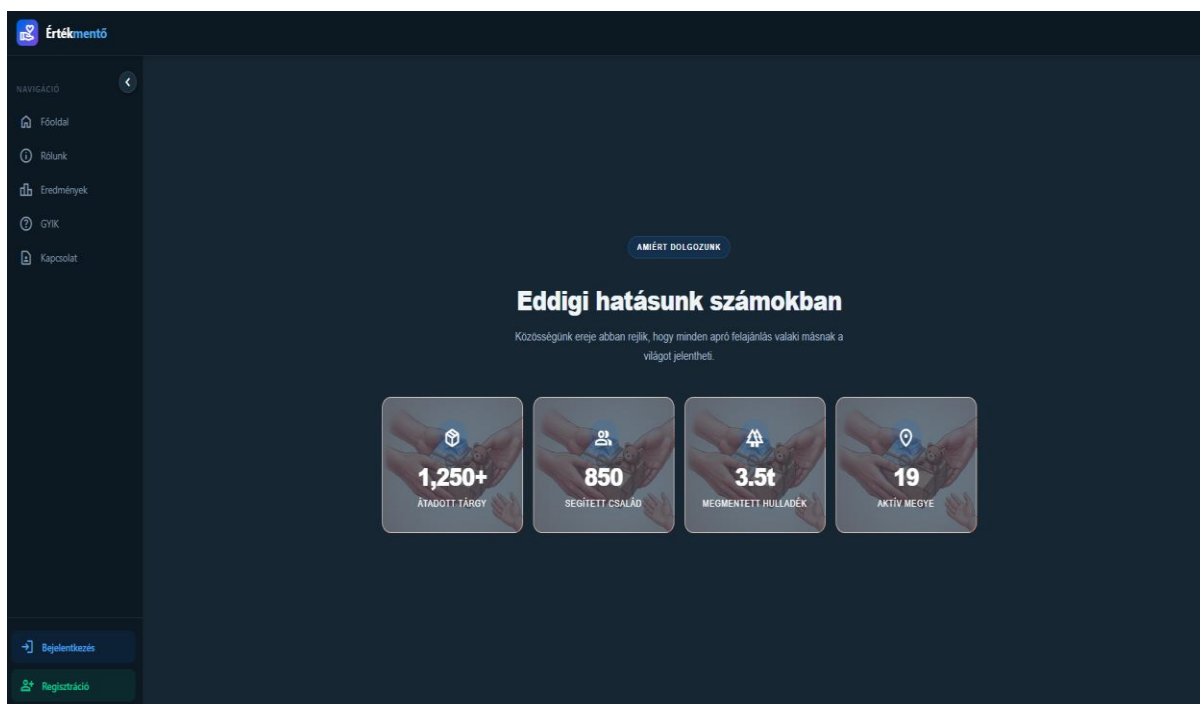
A lap tetején egy kétkártyás elrendezés látható mely mellett egy oldalsó navigációs sáv helyezkedik el, ami az egész főoldalon végig kísér. A bal oldali kártyán az „Adni szeretnék” opció található, amelyen egy hirdetés feladása gomb helyezkedik el. A jobb oldali kártyán a „Keresek valamit” opció látható, amelyen egy Termékek böngészése gomb navigál az elérhető adományok listájára. Bejelentkezés előtt ezek a gombok a bejelentkezési oldalra irányítanak.

3.4.2 Küldetésünk szekció



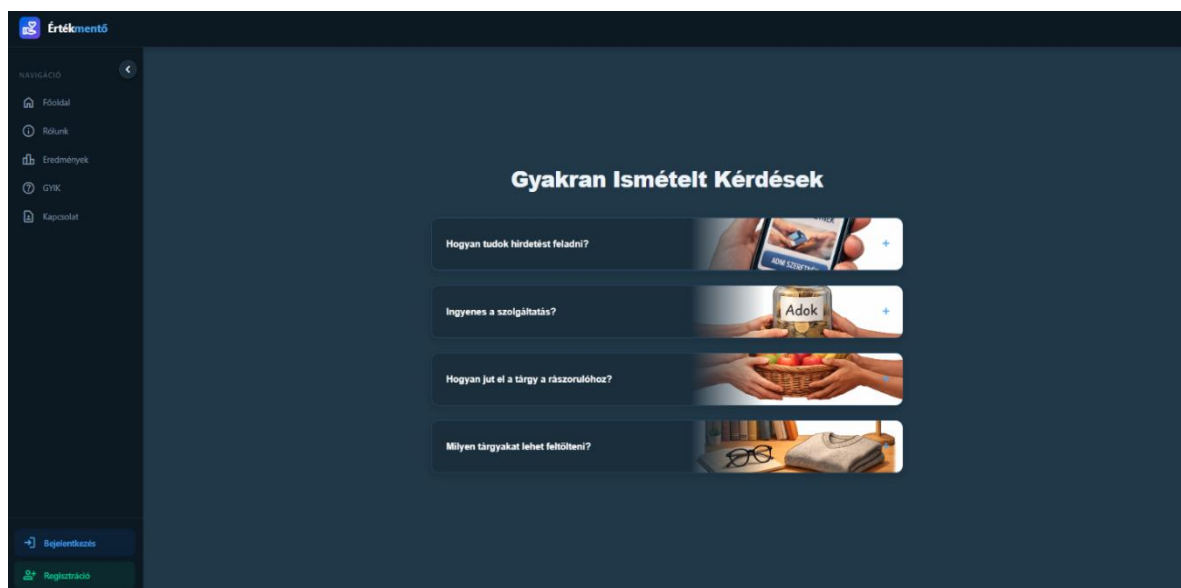
A miért jöttünk rész röviden bemutatja az Értékmentő célját és lét funkcióit: felesleges, de jó állapotú tárgyak eljuttatása azokhoz, akiknek szükségük van rájuk. A szekció felsorolja a platform három fő értékét: környezettudatos újrahasznosítás, közvetlen és gyors segítségnyújtás, ellenőrzött közösség.

3.4.3 Eredmények szekció



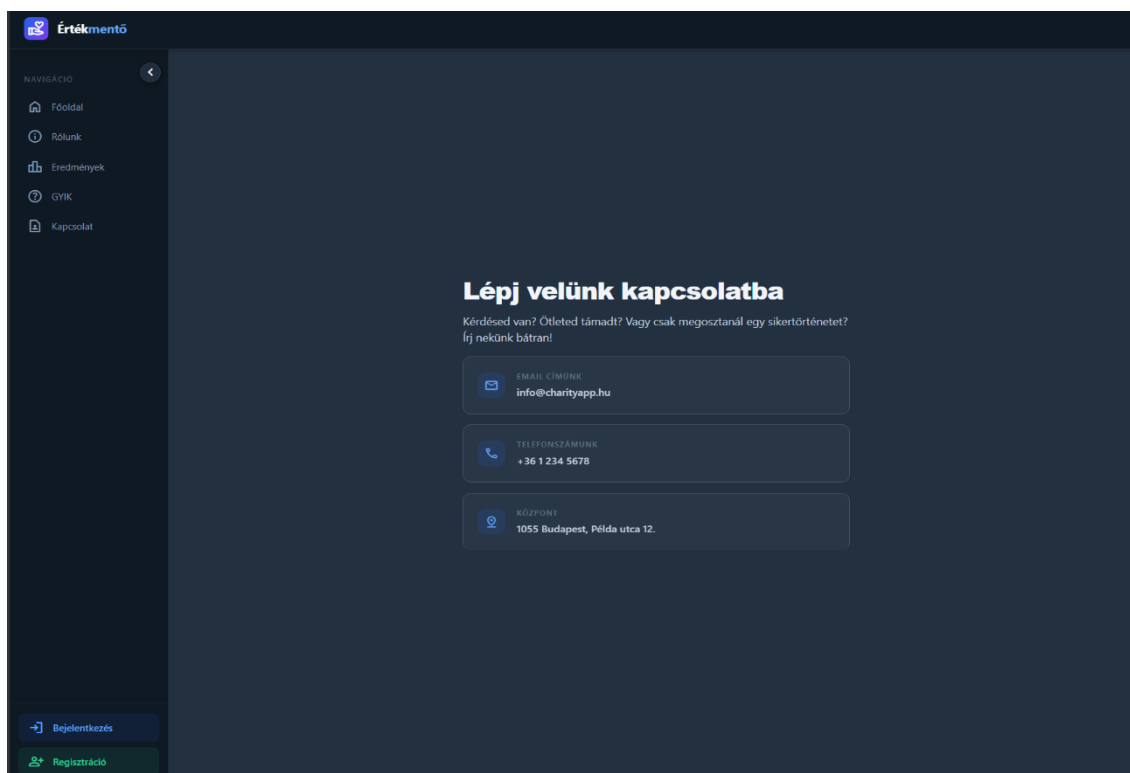
A „Eddigi hatásunk számokban” szekció négy mutató kártyán keresztül szemlélteti a platform eddigi hatását: az átadott tárgyak száma, a segített családok száma, a megmentett hulladék mennyisége, és az aktív vármegyék száma segítségével. Ezek a számok csak tájékoztató jellegű, nem valós adatokat tükröznek melyek betekintést engednek a felhasználók informálásában az oldal statisztikáiról.

3.4.4 Gyakran Ismételt Kérdések (GYIK)



A GYIK szekció legördülő panel-elemekben válaszolja meg a leggyakoribb kérdéseket: Hogyan lehet hirdetést feladni? Ingyenes-e a szolgáltatás? Hogyan jut el a tárgy a rászorulóhoz? Milyen tárgyakat lehet feltölteni? A panelek kattintásra nyílnak ki és adnak egy rövid kifejtést a kérdések megválaszolására.

3.4.5 Kapcsolati szekció



Az oldal alján található kapcsolatfelvételi szekció, melyben e-mail-cím (info@ertekmentoapp.hu), telefonszám (+36 1 234 5678) és cím szerepel. Ezen felül egy kapcsolati űrlap is elérhető, aminek segítségével fel lehet venni a kapcsolatot az ügyfélszolgálattal.

3.5 Regisztráció

The screenshot shows the 'Új fiók létrehozása' (Create new account) form within the 'Értékmentő' application. The form is displayed on a dark-themed interface with a sidebar on the left containing navigation links: 'Főoldal', 'Rólunk', 'Eredmények', 'GYK', and 'Kapcsolat'. The registration form itself is a light gray box with the following fields and labels:

- FELHASZNÁLÓNÉV**: Kérlek add meg a felhasználó neved
- VEZETÉKNÉV**: Kérlek add meg a vezetéknéved
- KERESZTNÉV**: Kérlek add meg a keresztnéved
- EMAIL CÍM**: Kérlek add meg az email címed
- JELSZÓ**: Kérlek add meg a jelszavad minimum 6
- JELSZÓ ÚJRA**: Kérlek add meg a jelszavad újból
- REGISZTRÁCIÓ CÉLJA**: -- Válassz szerepkört --

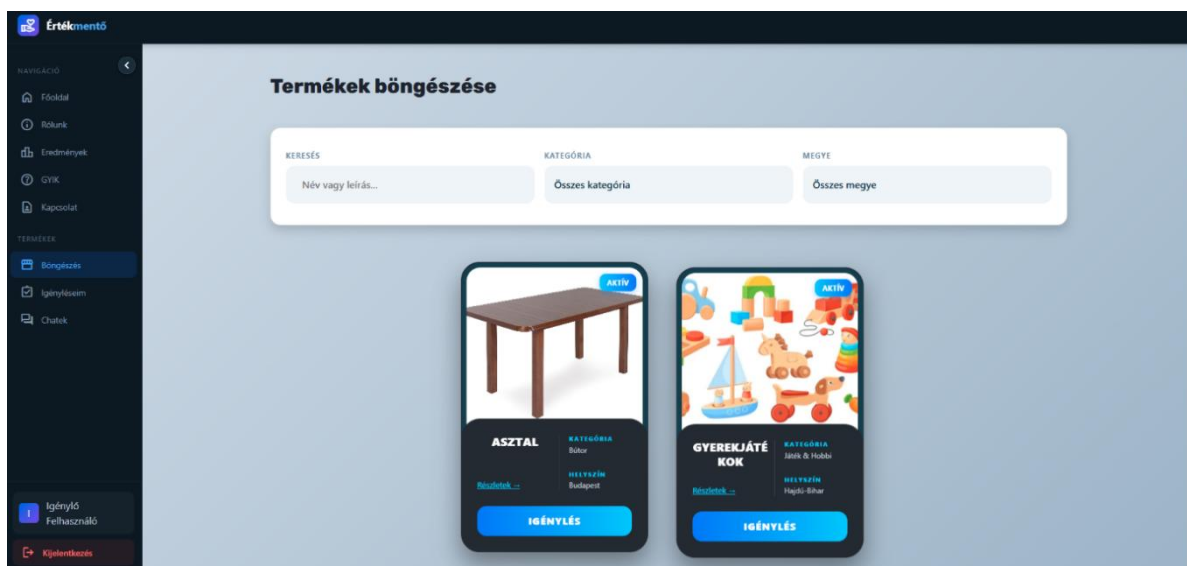
A 'Regisztráció' button is located at the bottom of the form.

A legtöbb funkció eléréséhez regisztrációra van szükség. A regisztrációs oldalt a navigációs sávban a „Regisztráció” gombbal teheti meg a felhasználó. A rendszer a regisztráció során bekéri a legfontosabb adatokat, majd ezek ellenőrzése után létrehozza a felhasználói profilt. A felület kialakításánál fontos szempont volt az egyszerűség és az egyértelmű adatbevitel támogatása. Ennek köszönhetően a regisztrációs folyamat gyorsan és különösebb technikai ismeret nélkül is elvégezhető.

3.5.1 Bejelentkezés

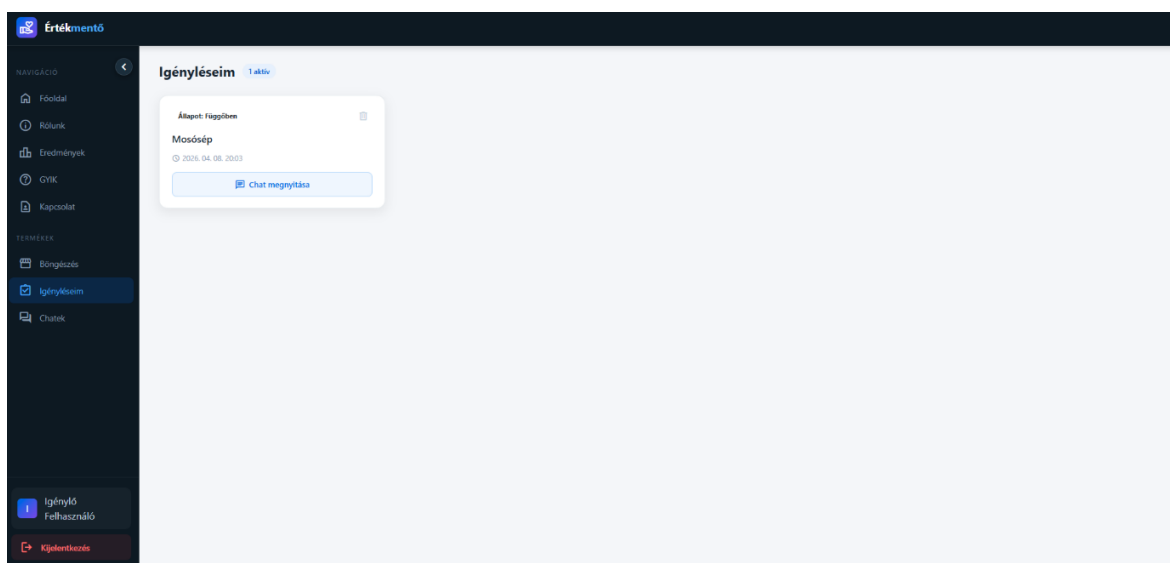
Bejelentkezés során a már regisztrált felhasználók hozzáférhetnek saját fiókjukhoz. A belépés a megadott azonosító adatok ellenőrzése után történik, amely biztosítja a személyes adatok és a felhasználói műveletek védelmét. A felület letisztult kialakítása segíti a gyors és hibamentes használatot. A rendszer a hibás adatok esetén megfelelő visszajelzést is ad a felhasználónak.

3.5.2 Igénylő felhasználó termékek böngészése felület



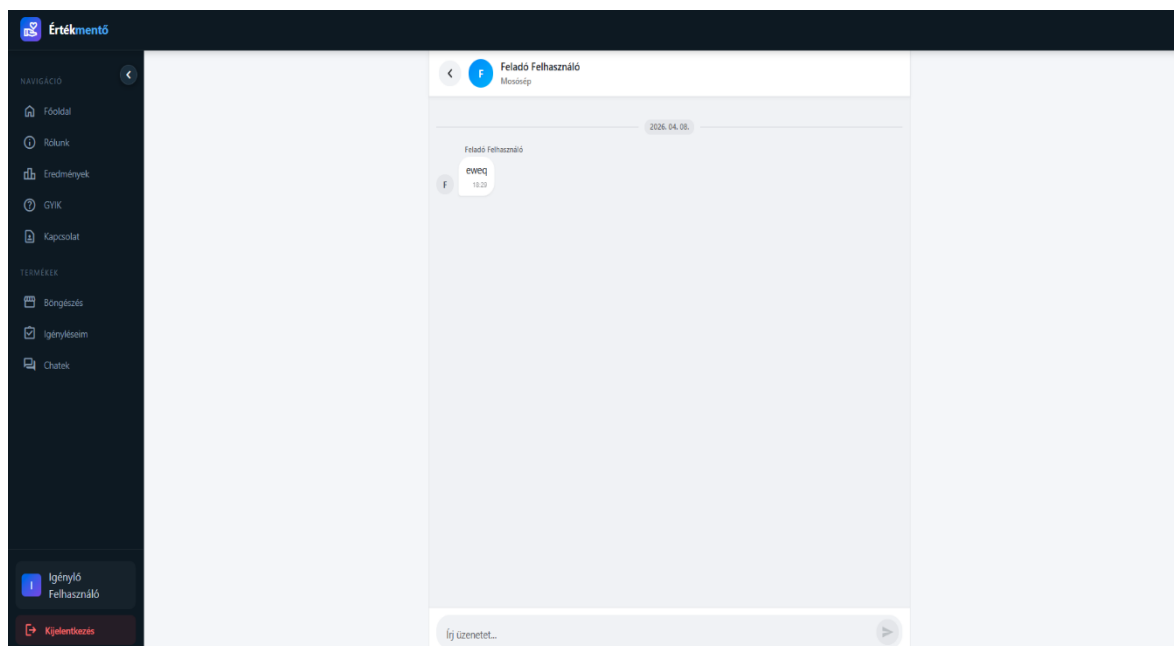
A termékek böngészése oldalon az igénylők át tekinthetik az eltérő felajánlásokat. A rendszer lehetőséget biztosít különböző szűrések használatára ennek segítségével a felhasználó gyorsabban meg tudja találni a számára releváns termékeket. A vizuális megjelenítés célja, hogy a termékek könnyen összehasonlíthatók és áttekinthetők legyenek. Ez a rész az alkalmazás egyik legfontosabb eleme.

3.5.3 Igénylő igénylései



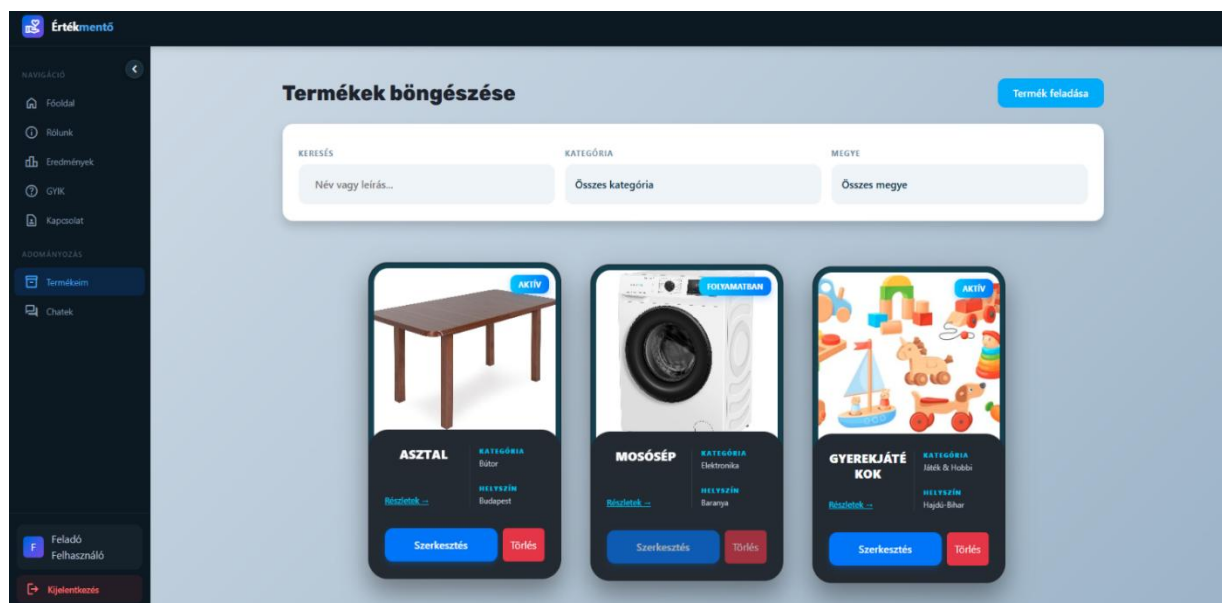
Az ábrán az igénylő felhasználó saját igényléseit megjelenítő oldal látható. Ez a felület összegzi azokat a termékeket, amelyekre a felhasználó korábban igényt nyújtott be, valamint megjeleníti azok aktuális állapotát is. A „Chat megnyitása” gomb lehetővé teszi a közvetlen kapcsolattvételt az adományozóval. Ez a megoldás átláthatóvá teszi az igénylési folyamat követését.

3.5.4 Igénylő Felhasználó Chat felülete



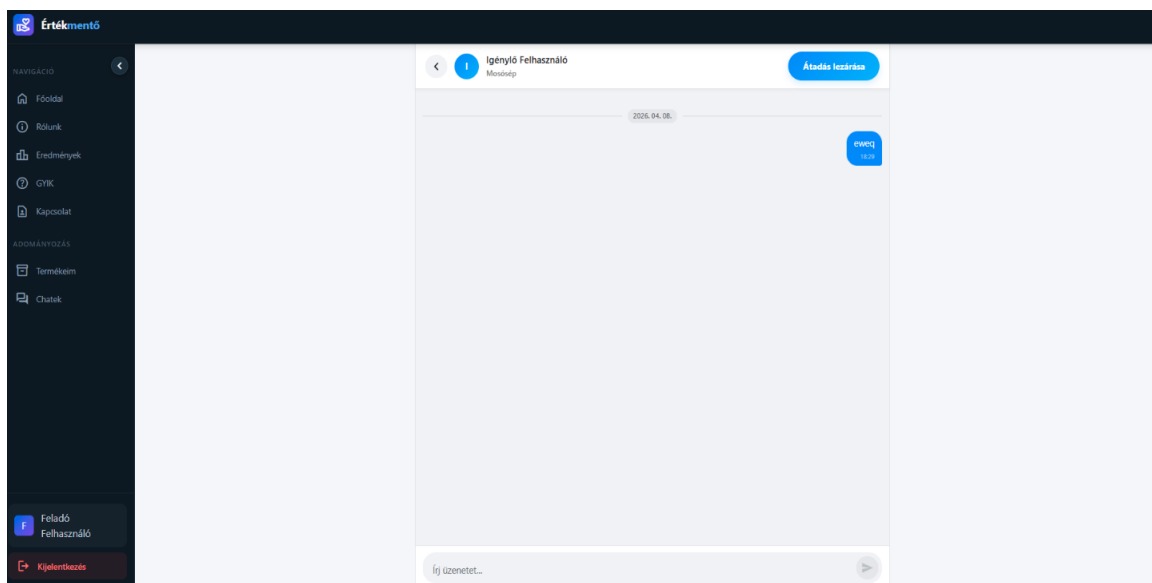
A felhasználó chat felületén keresztül az igénylő kommunikálhat a felajánlóval a termék átadásával kapcsolatban. Az üzenetek időrendben jelennek meg, így a korábbi kommunikáció is könnyen visszakövethető. A chatfunkció célja az, hogy egyszerűvé és gyorsá tegye az egyeztetést a felek között.

3.5.5 Felajánló termékei



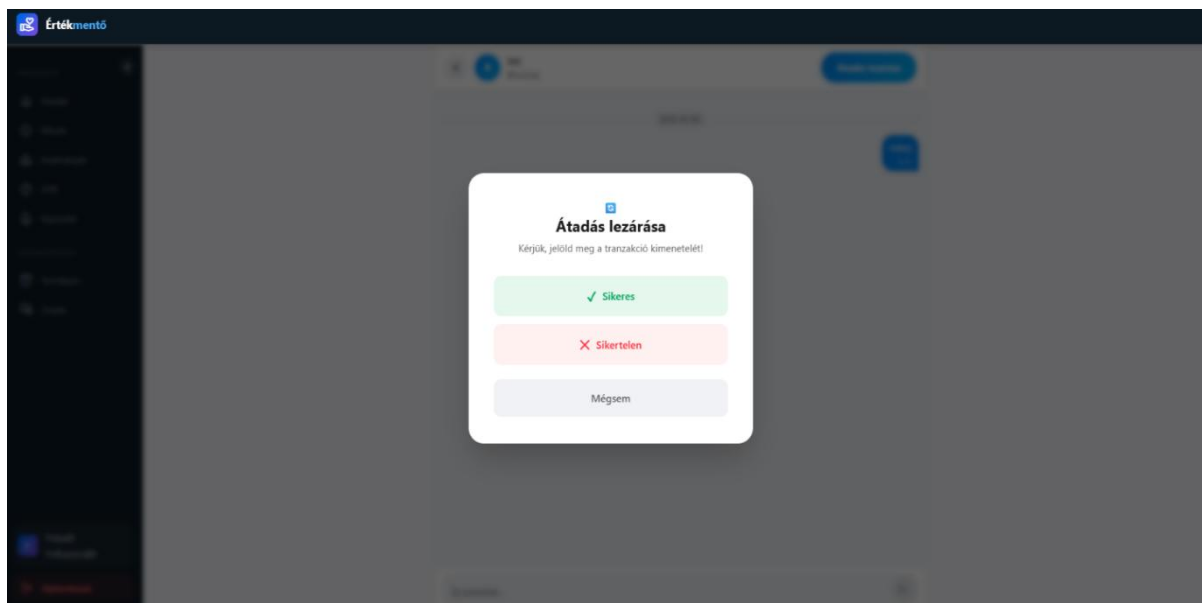
A felajánló termékei láthatók. Itt a felajánló a termék feltöltése gomb segítségével képes új termékeket feltölteni. A termék kártyákon pedig a szerkesztés és a törlés gombok segítségével képesek a termékeik adatot változtatni pl. név, kép. A törlés gombbal pedig levenni a kínálatból.

3.5.6 Felajánló chat felülete



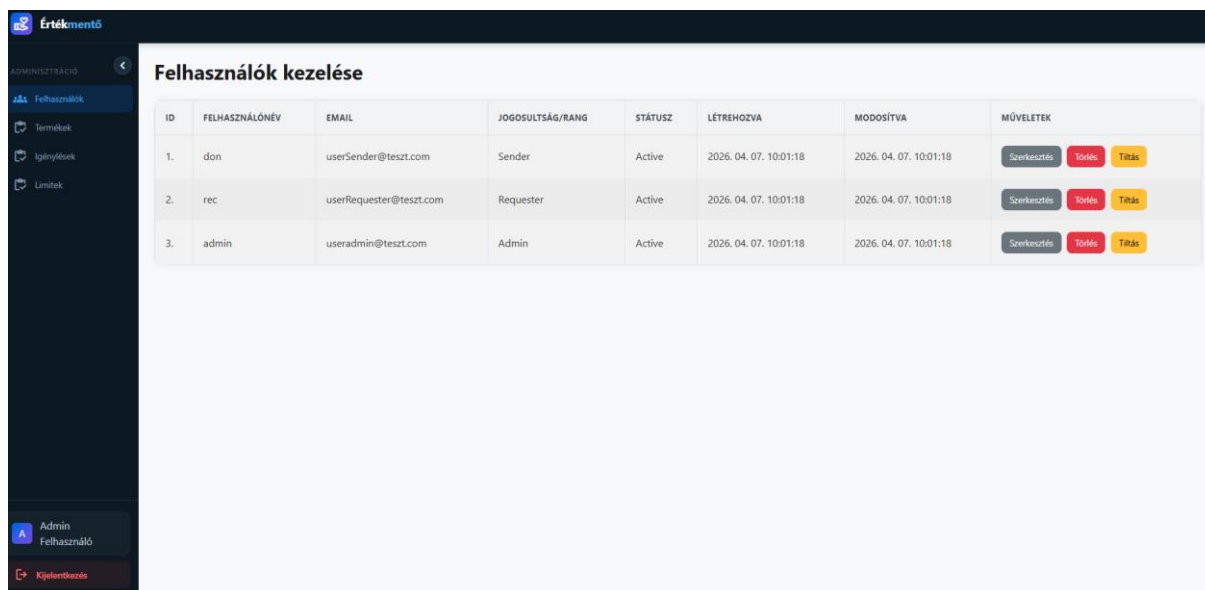
Az ábra az adományozó szemszögéből mutatja be a chatfelületet. Itt a felajánló nyomon követheti az igénylővel folytatott kommunikációt, és egyeztethet a termék átadásának részleteiről. A képernyő tetején látható a beszélgetéshez tartozó termék neve, amely segíti az azonosítást. Ez a funkció támogatja a közvetlen, gyors és hatékony kapcsolatfelvételt a két fél között.

3.5.7 Felajánló igénylés lezárása



A képen az átadás lezárását lehetővé tevő felugró ablak látható. A rendszer ezen a ponton megkéri a felhasználót, hogy jelölje meg a tranzakció kimenetelét, vagyis azt, hogy az átadás sikeres vagy sikertelen volt-e a mégse gomb pedig segít elkerülni az esetleges véletlen kattintásokat. Ez a visszajelzés fontos az igénylések állapotának naprakész kezeléséhez. A funkció hozzájárul ahhoz, hogy a rendszerben tárolt adatok pontosan tükrözzék a valós folyamatokat.

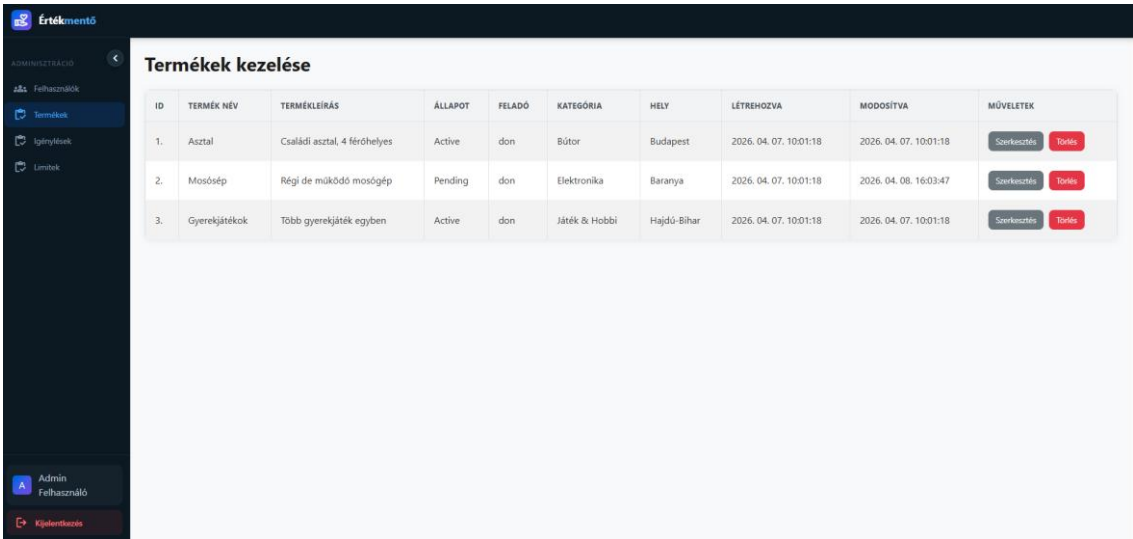
3.5.8 Admin felhasználó felhasználók kezelése



ID	FELHASZNÁLÓNÉV	EMAIL	JOGOSULTSÁG/RANG	STÁTUSZ	LÉTREHOZVA	MODOSÍTVA	MŰVELETEK
1.	don	userSender@teszt.com	Sender	Active	2026. 04. 07. 10:01:18	2026. 04. 07. 10:01:18	<button>Szerkesztés</button> <button>Törölés</button> <button>Tiltás</button>
2.	rec	userRequester@teszt.com	Requester	Active	2026. 04. 07. 10:01:18	2026. 04. 07. 10:01:18	<button>Szerkesztés</button> <button>Törölés</button> <button>Tiltás</button>
3.	admin	useradmin@teszt.com	Admin	Active	2026. 04. 07. 10:01:18	2026. 04. 07. 10:01:18	<button>Szerkesztés</button> <button>Törölés</button> <button>Tiltás</button>

A képen az adminisztrátorok számára kialakított felhasználókezelő oldal látható. A táblázatban megjelennek a regisztrált felhasználók adatai, pl: a felhasználónév, e-mail-cím, jogosultság és státusz. A műveleti gombok lehetőséget biztosítanak a felhasználók szerkesztésére, törlésére vagy tiltására. Ez a felület fontos eszköze az adminisztratív ellenőrzésnek és a rendszer felügyeletének.

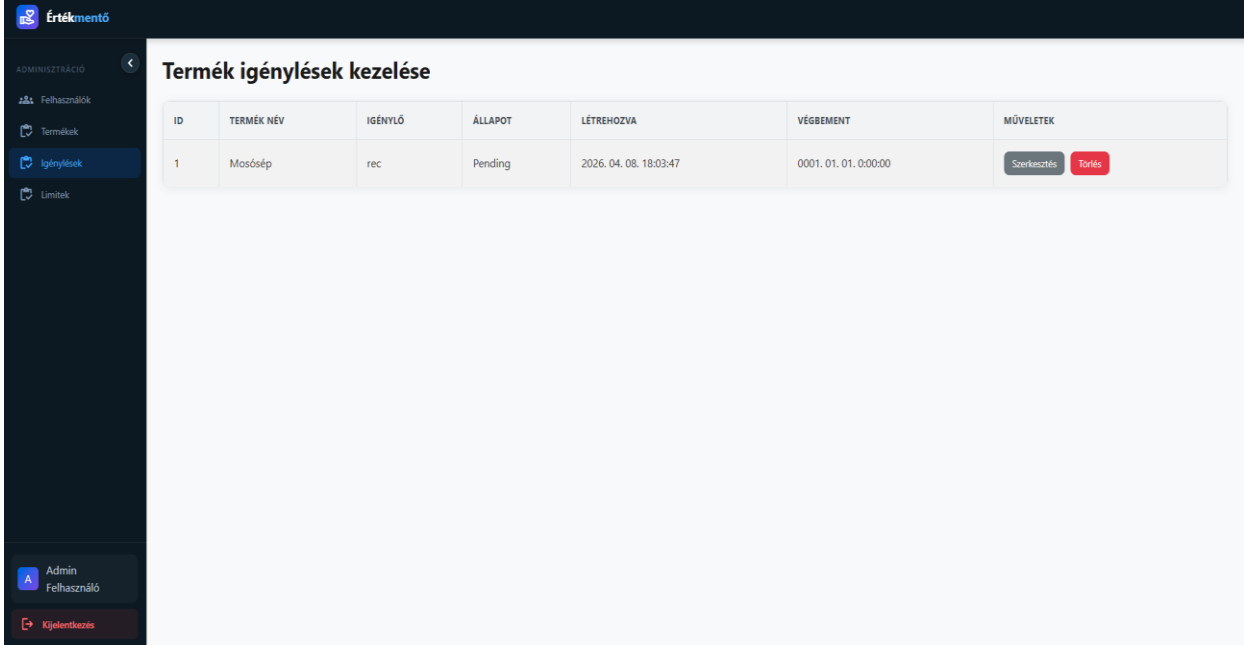
3.5.9 Admin felhasználó: termékek kezelésének felülete



ID	TERMÉK NÉV	TERMÉKLEÍRÁS	ÁLLAPOT	FELADÓ	KATEGÓRIA	HELY	LÉTREHOZVA	MODOSÍTVA	MŰVELETEK
1.	Asztal	Családi asztal, 4 férőhelyes	Active	don	Bútor	Budapest	2026. 04. 07. 10:01:18	2026. 04. 07. 10:01:18	Szerkesztés Törölés
2.	Mosógép	Régi de működő mosógép	Pending	don	Elektronika	Baranya	2026. 04. 07. 10:01:18	2026. 04. 08. 16:03:47	Szerkesztés Törölés
3.	Gyerekjátékok	Több gyerekjáték egyben	Active	don	Játék & Hobby	Hajdú-Bihar	2026. 04. 07. 10:01:18	2026. 04. 07. 10:01:18	Szerkesztés Törölés

Az ábrán az adminisztrátorok által használt termékkezelő oldal látható. Ezen a felületen az összes felajánlott termék áttekinthető, beleértve azok nevét, leírását, kategóriáját, helyét és állapotát. Az adminisztrátor innen közvetlenül tudja szerkeszteni vagy törölni az egyes termékeket. Ez lehetővé teszi a rendszer tartalmának központi ellenőrzését és karbantartását.

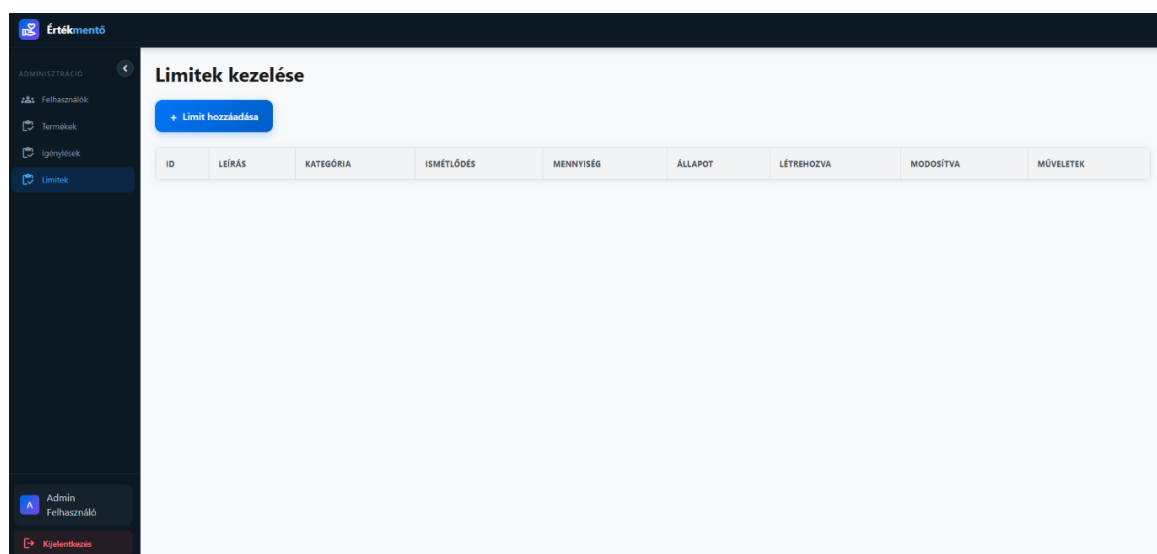
3.5.10 Admin felhasználó: igénylések kezelésének felülete



ID	TERMÉK NÉV	IGÉNYLŐ	ÁLLAPOT	LÉTREHOZVA	VÉGBEMENT	MŰVELETEK
1	Mosósép	rec	Pending	2026. 04. 08. 18:03:47	0001. 01. 01. 0:00:00	<button>Szerkesztés</button> <button>Töröl</button>

A képen az adminisztrációs felület igényléskezelő nézete látható. A táblázat segítségével az adminisztrátor nyomon követheti, hogy mely termékekre érkezett igénylés, ki volt az igénylő, valamint milyen állapotban van a folyamat. A felület támogatja az egyes igénylések szerkesztését és kezelését is. Ez különösen fontos a rendszer átlátható és ellenőrizhető működésének biztosításához.

3.5.11 Admin felhasználó: limitek kezelésének felülete



Az ábrán a limitszabályok kezelésére szolgáló felület figyelhető meg. Ezen az oldalon az adminisztrátor új limitet hozhat létre, illetve áttekintheti a már meglévő szabályokat. A limitszabályok célja az, hogy meghatározzák, egy felhasználó adott időszak alatt hány terméket igényelhet egy adott kategóriában. Ez segít megelőzni a visszaéléseket, és igazságosabbá teszi a rendszer működését.

3.5.12 A regisztrációs mezők leírása és elvárások

Mező neve	Mit kell ide írni?	Elvárások és szabályok
Vezetéknév	A felhasználó vezetéckneve	Csak betűk és kötőjel engedélyezett.
Keresztnév	A felhasználó keresztnéve	Csak betűk és kötőjel engedélyezett.
Felhasználónév	Egy egyedi, bejelentkezéshez használt azonosító	Csak betűk, számok és aláhúzásjel (_) engedélyezett. Nem tartalmazhat szóközt. Az egyediség ellenőrzésre kerül – ha a felhasználónév már foglalt, a rendszer jelzi.

E-mail-cím	Érvényes e-mail-cím	Kötelező. Érvényes e-mail formátum szükséges (pl. valaki@email.hu). Egyedinek kell lennie – ha az e-mail-cím már regisztrált, a rendszer jelzi.
Jelszó	Egy biztonságos jelszó	Kötelező. Minimum 6 karakter, tartalmaznia kell legalább 1 számot és legalább 1 betűt. Kis- és nagybetű megkülönböztetett.
Jelszó megerősítése	Ugyanaz a jelszó, mint fentebb	Kötelező. Pontosán meg kell egyeznie a jelszó mezőben megadott értékkel.
Szerepkör	Adományozó vagy Igénylő	Kötelező. Válassza ki, hogy adományokat szeretne feltölteni (Adományozó / Sender), vagy adományokat szeretne igényelni (Igénylő / Requester).

3.5.13 Lehetséges hibaüzenetek regisztráció során

Hibaüzenet	Oka	Megoldás
„A regisztráció nem sikerült. Lehet, hogy már létezik a felhasználó.”	A megadott e-mail-cím már szerepel az adatbázisban.	Próbáljunk másik e-mail-címet, vagy kattintsunk a „Bejelentkezés” linkre.
„A regisztráció nem sikerült. Lehet, hogy már létezik a felhasználó.”	A felhasználónév már foglalt.	Adjunk meg más felhasználónevet.
„A két jelszó nem egyezik.”	A jelszó és a megerősítés mezők eltérnek.	Írjuk be pontosan ugyanazt a jelszót mindkét mezőbe.

„A jelszónak legalább 6 karakter hosszúnak kell lennie.”	A jelszó túl rövid.	Adjunk meg legalább 6 karakteres jelszót.
„Kötelező mező.”	Egy kötelező mezőt üresen hagytunk.	Töltjük ki az összes piros kerettel jelölt mezőt.

3.6 Bejelentkezés

Ha már rendelkezünk regisztrált fiókkal, a bejelentkezési felületen tudunk belépni. A bejelentkezési oldalra a navigációs sávban a „Bejelentkezés” gombra kattintva jutunk.

3.6.1 A bejelentkezés lépései

32. Nyissuk meg a bejelentkezési oldalt.
33. Az „E-mail vagy Felhasználónév” mezőbe írjuk be a regisztrációkor megadott e-mail-títnket VAGY felhasználónevünket.
34. A „Jelszó” mezőbe írjuk be a regisztrációkor beállított jelszavunkat. Ha el szeretnénk ellenőrizni, amit beírtunk, kattintsunk a mező jobb oldalán lévő szem ikonra – ez láthatóvá teszi a jelszót. Újabb kattintásra ismét eltakaráó karakterekre vált.
35. Kattintsunk a „Belépés” gombra.
36. Helyes adatok esetén a rendszer a főoldalra irányít, és megjelenik a felhasználónév a navigációs sávban.

3.6.2 Bejelentkezési hibaüzenetek

Hibaüzenet	Oka és megoldása
„Hibás adatok.”	A megadott e-mail/felhasználónév és jelszó kombináció nem egyezik az adatbázisban tároltakkal. Ellenőrizzük, hogy nincs-e elgépelés, és hogy a Caps Lock nincs-e bekapcsolva.

„Hálózati hiba történt.”	A szerver nem elérhető. Ellenőrizzük az internetkapcsolatunkat, és próbáljuk újra később.
„User not active”	A fiók adminisztrátori döntés alapján felfüggesztett vagy tiltott. Vegyük fel a kapcsolatot az adminisztrátorral.

3.7 Termékek böngészése (Igénylői nézet)

A Termékek böngészése oldal az igénylők számára mutatja az összes aktív, igényelhető adományt. Az oldalra a főoldalon a „Termékek böngészése” gombbal, vagy a navigációs menüből lehet eljutni. Az oldal csak bejelentkezett Requester (igénylő) szerepkörű felhasználók számára elérhető.

3.7.1 Szűrési lehetőségek

Az oldal tetején egy szűrőkártya látható, amely háromféle szűrési lehetőséget kínál, amelyek kombinálhatók egymással:

- Keresés szabad szöveggel: A termék neve és leírása alapján szűr. Elég néhány betűt begépelni – az oldal valós időben frissíti a találatokat. A keresés nem érzékeny a kis- és nagybetűkre. Ékezetes betűk nélkül is működik (pl. „asztal” megtalálja az „Asztal” nevű terméket is).
- Kategória szerinti szűrés: A legördülő listából kiválasztva csak az adott kategóriába tartozó termékek jelennek meg. Alapesetben az „Összes kategória” van kijelölve.
- Megye szerinti szűrés: A legördülő listából kiválasztva csak az adott vármegyéből érkező felajánlások jelennek meg. Alapesetben az „Összes megye” van kijelölve.

3.7.2 Termékek megjelenítése

A szűrési feltételeknek megfelelő termékek kártyás formátumban jelennek meg, az újabb felajánlások előre rendezve. Minden kártyán látható a termék neve, kategóriája, vármegyéje és képe (ha van feltöltve). Ha 20-nál több termék felel meg a feltételeknek, a lapozó (pagination) vezérlő segítségével lehet a termékek között navigálni.

3.7.3 Termék részleteinek megtekintése

Egy termék kártyájára kattintva egy felugró panel (modal) jelenik meg, amely részletesebb leírást, a felajánló felhasználónevét és az igénylés gombját tartalmazza. A „Termék igénylése” gombra kattintva a rendszer igénylést nyújt be az adott termékre.

3.8 Termék felajánlása (Adományozói nézet)

Az adományozók a „Saját termékeim” oldalon tudnak új terméket feltölteni. Az oldal csak bejelentkezett Sender (adományozó) szerepkörű felhasználók számára érhető el.

3.8.1 Új felajánlás létrehozásának lépései

37. Navigáljunk a „Saját termékeim” oldalra (navigációs menü → Saját termékeim).
38. Kattintsunk az „Új termék feltöltése” gombra.
39. A megjelenő panelen töltsük ki az összes szükséges mezőt (ld. lent).
40. Opcionálisan töltsünk fel egy fényképet a termékről a kép feltöltési mezőn keresztül.
41. Kattintsunk a „Mentés” gombra. A termék azonnal megjelenik az igénylők számára.

3.8.2 A termék-feltöltési mezők leírása

Mező neve	Leírás	Elvárások
Termék neve	A felajánlott tárgy megnevezése	Kötelező. Maximum 100 karakter.
Leírás	A termék állapotának, jellemzőinek részletes leírása	Opcionális, de erősen ajánlott. Szabad szöveg, korlátlan hosszban.

Kategória	Melyik kategóriába tartozik a tárgy?	Kötelező. Válasszunk a legördülő listából (Bútor, Elektronika stb.).
Vármegye	Melyik vármegyében vehetjük át a tárgyat?	Kötelező. Válasszunk a 20 vármegye közül.
Kép feltöltése	A termékről készített fénykép	Opcionális. Támogatott formátumok: JPG, JPEG, PNG. Maximum fájl méret: 5 MB. Egy kép tölthető fel termékenként. A kép feltöltés után a szerveren kerül tárolásra.

3.9 Igénylések kezelése

3.9.1 Igénylői nézet – Saját igényléseim

A „Saját igényléseim” oldalon az igénylő megtekintheti az összes általa beadott igénylés állapotát. Az igénylések három státuszban lehetnek: Függőben (várakozik az adományozó döntésére), Teljesítve (az átadás megtörtént) és Sikertelen (az adományozó elutasította vagy az átadás megghiúsult). A Függőben lévő igénylések melletti „Chat” gombra kattintva a felhasználó közvetlen üzenetváltásba kezdhet az adományozóval.

Egy Függőben lévő igénylés törölhető is – ebben az esetben a termék visszakerül az aktív (igényelhető) állapotba, és más felhasználók is igényelhetik.

3.9.2 Adományozói nézet – Beérkezett igénylések

Az adományozó a „Saját termékeim” oldalon megtekintheti, hogy melyik termékére érkezett igénylés. Az igénylés melletti „Chat” gombra kattintva az adományozó üzenhet az igénylőnek, és egyeztetetheti az átadás időpontját, helyszínét. Ha az átadás sikeresen megtörtént, az adományozó a „Lezárás – sikeres” gombra kattint, amely átállítja a termék és az igénylés státuszát Completed értékre, és a limit-felhasználás csökken. Ha az átadás megghiúsult, a „Lezárás – sikertelen” gomb a terméket visszaállítja Active státuszra.

3.10 Chat – Üzenetváltás

Az integrált chat-rendszer lehetővé teszi, hogy az adományozó és az igénylő közvetlenül kommunikáljon az átadás részleteiről anélkül, hogy elhagynák az alkalmazást. A chatablak az igénylésekhez kapcsolódik – minden igényléshez pontosan egy chat-szoba tartozik.

A chat-felületre navigálva a bal oldalon az összes aktív chat-szoba listája látható, az olvasatlan üzenetek számával együtt (piros számbadge). A kiválasztott szobában jobb oldalon a korábbi üzenetek időrendben jelennek meg, saját üzenetek jobbra igazítva, a másik fél üzenetei balra igazítva. Az üzenetek valós időben frissülnek (SignalR), így az új üzenetek azonnal megjelennek oldal-frissítés nélkül.

Üzenet küldéséhez írjuk be a szöveget az alsó beviteli mezőbe, majd kattintsunk az „Küldés” gombra vagy nyomjuk meg az Enter billentyűt.

3.11 Adminisztrátori felület

Az adminisztrátori felület kizárólag Admin szerepkörű bejelentkezett felhasználók számára elérhető, a navigációs sávban az „Admin” menüpont alatt. A felület négy fő kezelési területet tartalmaz:

3.11.1 Felhasználókezelés (Admin → Felhasználók)

Az adminisztrátor megtekintheti az összes regisztrált felhasználót, szerkesztheti adataikat, valamint tilthatja vagy törölheti fiókjukat. Egy tiltott felhasználó nem tud bejelentkezni. Admin-fiók létrehozásához a UserRole értékét 3-ra kell állítani az adatbázisban, vagy egy másik adminisztrátornak kell módosítania a felhasználó szerepkörét.

3.11.2 Termékkezelés (Admin → Termékek)

Az adminisztrátor megtekintheti az összes terméket (aktív, törölten és lezártat egyaránt), szerkesztheti adataikat, és törölheti azokat. Ez a funkció moderációs célt szolgál – ha egy termék leírása nem megfelelő tartalmat tartalmaz, az adminisztrátor beavatkozhat.

3.11.3 Igényléskezelés (Admin → Igénylések)

Az adminisztrátor megtekintheti az összes igénylés aktuális állapotát, szerkesztheti és törölheti azokat. Ez különösen hasznos, ha egy igénylés valamilyen technikai okból elakadt és manuális beavatkozás szükséges.

3.11.4 Limitszabály-kezelés (Admin → Limitszabályok)

Ez a felület az igénylési limitszabályok kezelésére szolgál. Az adminisztrátor létrehozhat, szerkeszthet és törölhet limitszabályokat. Új szabály létrehozásakor meg kell adni:

Kategória: melyik termékkategóriára vonatkozik a szabály

Időszak típusa: Weekly (heti), Monthly (havi), Quarterly (negyedéves) vagy Semiannual (féléves)

Maximum mennyiség: hány terméket igényelhet egy felhasználó a megadott időszakban

Leírás: opcionális megjegyzés a szabályhoz

Például: ha az adminisztrátor egy „Bútor – Havi 2 db” szabályt hoz létre, akkor egyetlen igénylő sem igényelhet 2-nél több bútort egy naptári hónapon belül.

3.12 Kijelentkezés

A fiókból való kilépéshez kattintsunk a navigációs sávban a felhasználónévre vagy a „Kijelentkezés” gombra. A rendszer törli a böngészőben tárolt autentikációs token-t, és visszairányít a főoldalra. A kijelentkezés után a bejelentkezést igénylő oldalak nem lesznek elérhetők.

3.13 Regisztráció nélkül elérhető funkciók

Funkció	Elérhető regisztráció nélkül?	Elérhető bejelentkezés után?
Főoldal megtekintése, GYIK, Kapcsolat	IGEN	IGEN
Termékek böngészése	NEM	IGEN (Igénylő szerepkör)
Termék igénylése	NEM	IGEN (Igénylő szerepkör)
Termék feltöltése	NEM	IGEN (Adományozó szerepkör)
Chat-kommunikáció	NEM	IGEN (mindkét szerepkör)
Adminisztrátori felület	NEM	IGEN (Admin szerepkör)

3.14 Elérhetőség, információkérés

Amennyiben kérdés, probléma vagy javaslat merül fel az alkalmazással kapcsolatban, az alábbi elérhetőségeken lehet felvenni a kapcsolatot:

- E-mail: kemey.benedek.mate@ganziskola.hu
- E-mail: papp.bence@ganziskola.hu
- Válaszadási határidő: legkésőbb 3 munkanapon belül.

4. Összefoglalás

4.1 Az elkészült munka értékelése

Az Értékmentő fejlesztése során sikerült megvalósítani a tervezett funkciókat: felhasználó-hitelesítés JWT-vel, szerepköralapú hozzáférés, termékfeltöltés képpel, valós idejű chat SignalR-ral és adminisztrátori limitkezelés.

A legnagyobb kihívást a Clean Architecture következetes alkalmazása jelentette – az egyes rétegek elszigeteléséhez és az interfészek megtervezéséhez több nekifutás kellett. A Unit of Work és a Generic Repository minta először bonyolultabbnak tűnt a szükségesnél, de végül megkönnyítette a kód karbantartását.

A SignalR integrálása a Blazor Server renderelési modelljével szintén sok hibakeresést igényelt, főleg a chat-szobák szinkronizálásánál. Az alkalmazás jelenlegi állapotában működőképes, de éles üzemhez a 2.6-os fejezetben leírt fejlesztések szükségesek lennének.

4.2 Tanulságok és személyes fejlődés

A projekt során a legtöbbet talán az architektúra fontosságából tanultuk. Amikor először nekiláttunk a Clean Architecture rétegeit felépíteni, nem volt egyértelmű, miért kell ennyire szétválasztani a kódot – a Domain, Application, Infrastructure és API rétegek megkülönböztetése eleinte inkább bonyolítani látszott a munkát, mint segíteni. Csak akkor vált érthetővé az egész, amikor az egyik rétegben kellett változtatni, és kiderült, hogy a többi érintetlen marad.

A SignalR integrálása volt a fejlesztés legizgalmasabb és egyben legtöbb hibakeresést igénylő része. A Blazor Server renderelési modellje és a SignalR hub közötti szinkronizáció sokszor produkált olyan hibát, amelynek oka csak hosszas naplóolvasás után derült ki. Ez a folyamat megtanított arra, hogyan kell valós idejű rendszerekben gondolkodni: nem elég azt tudni, mi történik a szerveren – pontosan érteni kell, mikor és hogyan jut el az adat a klienshez.

Visszatekintve néhány dolgot csinálnánk másképp: hamarabb vettük volna be a tesztelést a fejlesztési folyamatba. A tesztek utólag való megírása sokkal több energiát emésztett fel, mint amennyibe folyamatos, fejlesztéssel párhuzamos tesztelés kerül. Valamint a projekt dokumentációját is időben el kellett volna kezdeni így strukturálisan könnyebb lett volna felépíteni. Ezt a következő projektnél már másképp fogjuk csinálni.

4.3 A program jövőbeli hasznosítása

Az Értékmentő platform a jelenlegi formájában is alkalmas arra, hogy valós szociális igényt elégítsen ki. A fejlesztők szándéka, hogy a projektet a vizsga után is aktívan fejlesszék, és éles üzembe helyezzék. Rövid távon cél a Docker-alapú deployment elvégzése, és egy valós hosting-szolgáltatón (pl. Azure App Service, Railway.app) való üzemeltetés. Középtávon a mobilalkalmazás fejlesztése és az e-mail értesítési rendszer integrálása a prioritás.

A projektet a vizsga után is szeretnénk tovább fejleszteni. Első lépésként Docker-alapú deployment-et tervezünk, hogy az alkalmazás egy valós hosting-szolgáltatón (pl. Railway.app) fusson. Utána az e-mail értesítések és egy mobilbarát felület következne. Hosszabb távon esetleg karitatív szervezetekkel is lehetne együttműködni, de ez egyelőre még csak ötlet.

4.4 Köszönetnyilvánítás

Köszönjük felkészítő tanárainknak a szakmai segítséget és türelmet. Köszönjük azoknak az ismerőseinknek, akik tesztelték az alkalmazást és visszajelzést adtak. Végül köszönjük szüleinknek és testvéreinknek a támogatást a fejlesztés időszaka alatt.

5. Irodalomjegyzék

5.1 Online dokumentációk és webes forrásanyagok

Az alábbiakban a fejlesztés során felhasznált webes forrásanyagok találhatók, teljes mélységű URL-ekkel megadva:

- ASP.NET Core Web API dokumentáció – <https://learn.microsoft.com/hu-hu/aspnet/core/web-api/>
- Entity Framework Core – Getting Started – <https://learn.microsoft.com/hu-hu/ef/core/get-started/overview/first-app>
- Blazor Server-Side dokumentáció – <https://learn.microsoft.com/hu-hu/aspnet/core/blazor/hosting-models>
- ASP.NET Core SignalR bevezetés – <https://learn.microsoft.com/hu-hu/aspnet/core/signalr/introduction>
- JWT.io – JSON Web Tokens dokumentáció – <https://jwt.io/introduction>
- BCrypt.Net-Next NuGet csomag leírása – <https://github.com/BcryptNet/bcrypt.net>
- SQL Server T-SQL dokumentáció – <https://learn.microsoft.com/hu-hu/sql/t-sql/language-reference>
- Bootstrap 5 dokumentáció – <https://getbootstrap.com/docs/5.3/>
- Swagger / OpenAPI specifikáció – <https://swagger.io/specification/>
- Repository pattern és Unit of Work (Microsoft) – <https://learn.microsoft.com/hu-hu/aspnet/mvc/overview/older-versions/getting-started-with-ef-5-using-mvc-4/implementing-the-repository-and-unit-of-work-patterns-in-an-asp-net-mvc-application>
- Dokumentáció szempontjai – <https://infojegyzet.hu/webszerkesztes/dokumentacio/>
- Ötletelés és hibafeltárás Chatgpt4 és Gemini

5.2 Szakmai könyvek

- Web API Development with ASP.NET Core 8 => Xiaodi Yan
- Pro ASP.NET Core 6 => Adam Freeman

6. Melléklet – DTO-k és interfészek részletes leírása

Az Értékmentő alkalmazásban az adatátviteli objektumok (DTO – Data Transfer Object) kulcsszerepet játszanak. Ezek az osztályok az API és a kliens között, valamint az Application és az API réteg között közvetítik az adatokat, anélkül, hogy a belső domain entitásokat exponálnák. Ebben a fejezetben minden DTO részletes ismertetése olvasható.

6.1 RegisterDto – Regisztrációs adatátviteli objektum

A RegisterDto az új felhasználó regisztrációjakor kerül felhasználásra. A Blazor kliensen kitöltött regisztrációs form adatait tartalmazza, amelyek az API POST /api/Auth/register végpontjára kerülnek elküldésre. Az objektum DataAnnotations attribútumokkal ellátott mezőket tartalmaz, amelyek kliens oldali validációt tesznek lehetővé.

Tulajdonság neve	Típus	Validációs szabályok	Leírás
UserName	string	[Required], [StringLength(50, MinimumLength=3)]	Egyedi felhasználónév, 3-50 karakter
Email	string	[Required], [EmailAddress]	Érvényes e-mail-formátum szükséges
Password	string	[Required], [MinLength(6)]	Minimum 6 karakteres jelszó
FirstName	string	[Required]	Keresztnév
LastName	string	[Required]	Vezetéknév

RoleId	int	[Required], értéke 1 vagy 2	1=Sender, 2=Requester szerepkör azonosítója
--------	-----	-----------------------------	---

6.2 LoginDto és LoginResponseDto

A LoginDto az API bejelentkezési végpontjára küldött adatokat tartalmazza. A LoginResponseDto a sikeres bejelentkezés után a kliensnek visszaküldött JWT tokent csomagolja be.

Tulajdonság	Típus	Megjegyzés
LoginDto.EmailOrUserName	string	E-mail-cím VAGY felhasználónév – mindkettő elfogadott
LoginDto.Password	string	A bejelentkezési jelszó plaintext formában (csak HTTPS-en küldve)
LoginResponseDto.Token	string	A generált JWT Bearer token, amellyel a kliens autentikálni tud

6.3 ProductDto – Termékadat-átviteli objektum

A ProductDto minden termékkel kapcsolatos API-műveletnél felhasználásra kerül: lekérdezésnél, létrehozásnál és szerkesztésnél egyaránt. Ez az egyetlen objektum, amelyet a szerver a termékek listázásakor visszaad, és amellyel a kliensre kerülő kártyák fel vannak töltve.

Tulajdonság neve	Típus	Írány	Leírás
ProductId	int	Lekérdezésnél szerver → kliens	Az adatbázis-beli egyedi azonosító
ProductName	string	Mindkét irányban	A termék neve
ProductDescription	string?	Mindkét irányban	Opcionális leírás
ImagePath	string?	Mindkét irányban	Kép relatív URL-je a szerveren (pl. /images/products/abc.jpg)
ProductCategoryId	int	Mindkét irányban	Kategória azonosítója (FK → ProductCategory)
CountyId	int	Mindkét irányban	Vármegye azonosítója (FK → Counties)
SenderId	int	Szerver → kliens	Az adományozó azonosítója (kliens nem adja meg)
ProductStatus	ProductStatus enum	Szerver → kliens	Termék állapota: Active, Pending, Completed, Deleted
CreatedAt	DateTime	Szerver → kliens	Feltöltés időpontja (szerkesztéshez)

6.4 ProductRequestDto – Igénylés adatátviteli objektum

A ProductRequestDto az igénylések listázásakor és kezelésekor kerül felhasználásra. Tartalmazza az igénylés azonosítóját, a vonatkozó termék és igénylő adatait, az állapotot, és az időbélyegeket.

Tulajdonság neve	Típus	Leírás
ProductRequestId	int	Egyedi igénylés-azonosító
ProductId	int	Az igényelt termék azonosítója
RequesterId	int	Az igénylő felhasználó azonosítója
RequestStatus	RequestStatus enum	Pending (1), Completed (2) vagy Failed (3)
RequestedAt	DateTime	Igénylés benyújtásának időpontja
ProcessedAt	DateTime?	Lezárás időpontja (null, ha még nyitott)

6.5 ChatMessageRequestDto és ChatMessageResponseDto

A ChatMessageRequestDto az üzenetküldési API-híváskor kerül felhasználásra. Tartalmazza a vonatkozó igénylés azonosítóját és az üzenet tartalmát. A ChatMessageResponseDto a chat előzményeinek lekérésekor kerül visszaküldésre, részletes metaadatokkal együtt.

Objektum	Tulajdonság	Típus	Leírás
ChatMessageRequestDto	RequestId	int	Az igénylés azonosítója, amelyhez az üzenet tartozik
ChatMessageRequestDto	Content	string	Az üzenet szöveges tartalma
ChatMessageResponseDto	Id	int	ChatMessageId az adatbázisból
ChatMessageResponseDto	SenderId	int	Az üzenetet küldő felhasználó azonosítója
ChatMessageResponseDto	SenderName	string	A küldő teljes neve (FirstName + LastName)
ChatMessageResponseDto	OtherPartyName	string	A másik fél neve (kontextus megjelenítéshez)
ChatMessageResponseDto	Content	string	Az üzenet tartalma
ChatMessageResponseDto	SentAt	DateTime	Küldés UTC időbélyege
ChatMessageResponseDto	IsRead	bool	Olvasta-e a másik fél

6.6 RequesterLimitRuleDto – Limitszabály adatátviteli objektum

A RequesterLimitRuleDto az adminisztrátori felületen a limitszabályok CRUD-műveleteiben kerül felhasználásra. Az IsActive mező lehetővé teszi a szabály ideiglenes letiltását anélkül, hogy törölni kellene.

Tulajdonság neve	Típus	Leírás
RequesterLimitRuleId	int	Azonosító (0, ha új létrehozáskor)
RequesterLimitRuleCategoryId	int	Melyik kategóriára vonatkozik
PeriodType	string	"Weekly", "Monthly", "Quarterly" vagy "Semiannual"
MaxQuantity	int	Maximum igényelhető darabszám az időszakban
RequesterLimitRuleDescription	string?	Opcionális leírás/megjegyzés
IsActive	bool	Aktív-e a szabály (false = szüneteltetett)

7. Melléklet – Az API végpontok teljes dokumentációja

Az Értékmentő RESTful API a következő fő erőforrás-csoportokra épül: hitelesítés, termékek, igénylések, chat, limit-kezelés, adminisztráció és referencia-adatok. Az alábbiakban minden végpont részletes leírása található, beleértve a szükséges autentikációt, a bemeneti és kimeneti formátumot.

7.1 AuthController – Hitelesítési végpontok

Metódus	Végpont URL	Hitelesítés	Bemenet	Kimenet	Leírás
POST	/api/Auth/register	Nyilvános	RegisterDto (JSON body)	200 OK / 400 Bad Request	Új felhasználó regisztrálása
POST	/api/Auth/login	Nyilvános	LoginDto (JSON body)	200 OK + Login ResponseDto / 401	Bejelentkezés, JWT token kiadása

7.2 ProductController – Termékkezelési végpontok

Metódus	Végpont URL	Hitelesítés	Leírás
GET	/api/Product	Nyilvános	Összes aktív termék lekérése. A válasz ProductDto-k tömbje.
GET	/api/Product/my-products	Bearer (Sender)	A bejelentkezett adományozó saját termékeinek lekérése (nem töröltek, nem lezártak).

POST	/api/Product	Bearer (Sender)	Új termék létrehozása képpel (multipart/form-data). A SenderId automatikusan a bejelentkezett felhasználó azonosítója.
PUT	/api/Product/update-product	Bearer	Termék szerkesztése (név, leírás, kategória, megye). Csak Active státuszú termék szerkeszthető.
DELETE	/api/Product/{id}	Bearer (Sender)	Termék „törlése” (Deleted státuszra állítás, tényleges törlés nem történik).

7.3 ProductRequestController – Igényléskezelési végpontok

Metódus	Végpont URL	Hitelesítés	Leírás
POST	/api/ProductRequest/claim/{productId}	Bearer (Requester)	Termék igénylése. Ellenőrzi a terméket, a limitet és az aktív igénylést.
GET	/api/ProductRequest/my-requests	Bearer (Requester)	A bejelentkezett igénylő összes igénylésének listázása.
DELETE	/api/ProductRequest/request/{requestId}	Bearer	Egy Pending igénylés törlése. Csak a saját igénylés törölhető.

GET	/api/ProductRequest /Sender-requests	Bearer (Sender)	Az adományozó termékeire beérkezett igénylések listázása.
POST	/api/ProductRequest/complete/{requestId}	Bearer (Sender)	Igénylés lezárása. ?success=true → Completed; ?success=false → Failed.
GET	/api/ProductRequest/active-for-product/{productId}	Bearer	Van-e aktív igénylés az adott termékre a bejelentkezett usertől? 200=igen, 404=nincs.
GET	/api/ProductRequest/is-claimed/{productId}	Bearer	Van-e bárki által aktív igénylés? 200=foglalt, 404=szabad.

7.4 ChatController – Chat végpontok

Metódus	Végpont URL	Hitelesítés	Leírás
POST	/api/Chat/send	Bearer	Üzenet küldése egy igényléshez. Ha a chatszoba még nem létezik, automatikusan létrehozza.
GET	/api/Chat/history/{requestId}	Bearer	Egy igénylés chat-előzményeinek lekérése időrendben (legrégebbi → legújabb).
GET	/api/Chat/info/{requestId}	Bearer	Chat meta-adatok lekérése: a másik fél neve, termék neve, küldő azonosítója.
GET	/api/Chat/unread-count	Bearer	Az összes olvasatlan üzenet számának lekérése a bejelentkezett felhasználónak.
POST	/api/Chat/mark-read/{requestId}	Bearer	Egy chat összes üzenetének olvasottként jelölése.

7.5 LimitController – Limit-kezelési végpontok

Metódus	Végpont URL	Leírás
GET	/api/Limit/can-request?userId={id}&categoryId={id}	Ellenőrzi, hogy a felhasználónak van-e még szabad kapacitása az adott kategóriában. Visszaad: { canRequest: true/false }
POST	/api/Limit/use?userId={id}&categoryId={id}	Egy egységnyi limit-felhasználás rögzítése. Visszaad: { success: true/false }

7.6 AdminController – Adminisztrátori végpontok

Metódus	Végpont URL	Leírás
GET	/api/Admin/users	Összes felhasználó lekérése
PUT	/api/Admin/ban-user/{id}	Felhasználó kitiltása (UserStatus=Banned)
PUT	/api/Admin/delete-user/{id}	Felhasználó törlése (UserStatus=Deleted)
PUT	/api/Admin/update-user	Felhasználói adatok szerkesztése (UserDto body)
GET	/api/Admin/products	Összes termék lekérése (Admin nézet)
PUT	/api/Admin/update-product	Termék szerkesztése admin jogkörrel
PUT	/api/Admin/delete-product/{id}	Termék törlése admin jogkörrel
GET	/api/Admin/product-requests	Összes igénylés lekérése
PUT	/api/Admin/update-product-requests	Igénylés szerkesztése
PUT	/api/Admin/delete-product-requests/{id}	Igénylés törlése
GET	/api/Admin/requester-limitrules	Összes limitszabály lekérése
POST	/api/Admin/create-requester-limitrule	Új limitszabály létrehozása

PUT	/api/Admin/update-requester-limitrule	Limitszabály szerkesztése
PUT	/api/Admin/delete-requester-limitrule/{id}	Limitszabály törlése (soft delete: IsActive=false)

7.7 LookupController – Referenciaadat-lekérések

Metódus	Végpont URL	Leírás
GET	/api/Lookup/product-categories	Összes aktív termékkategória lekérése (szűrőkhöz)
GET	/api/Lookup/counties	Összes vármegye lekérése (szűrőkhöz)
GET	/api/Lookup/users	Összes felhasználó lekérése (adminisztrátori célokra)
GET	/api/Lookup/products	Összes termék lekérése szűrés nélkül (adminisztrátori célokra)

8. Melléklet – Az adatbázis-séma teljes SQL-szkriptje és kapcsolatok

Ebben a fejezetben az adatbázis teljes létrehozási szkriptjének részletes elemzése található, különös tekintettel a táblák közötti kapcsolatokra, az indexekre és a kényszerekre.

8.1 Táblák közötti kapcsolatok

Az Értékmentő adatbázisában az alábbi idegen kulcs (FOREIGN KEY) kapcsolatok definiálnak összefüggéseket a táblák között:

Gyermek tábla	Gyermek mező	Szülő tábla	Szülő mező	ON DELETE szabály
Products	SenderId	Users	UserId	NO ACTION (alapértelmezett)
Products	ProductCategoryId	ProductCategory	ProductCategoryId	NO ACTION
Products	CountyId	Counties	CountyId	NO ACTION
ProductRequests	ProductId	Products	ProductId	CASCADE DELETE
ProductRequests	RequesterId	Users	UserId	NO ACTION

RequesterLimitRule	RequesterLimitRuleCategoryId	ProductCategory	ProductCategoryId	NO ACTION
RequesterLimitUsage	RequesterId	Users	UserId	NO ACTION
RequesterLimitUsage	RuleId	RequesterLimitRule	RequesterLimitRuleId	NO ACTION
Chat	ProductRequestId	ProductRequests	ProductRequestId	CASCADE DELETE
Chat	SenderId	Users	UserId	NO ACTION
Chat	RequesterId	Users	UserId	NO ACTION
ChatMessage	ChatId	Chat	ChatId	CASCADE DELETE
ChatMessage	SenderId	Users	UserId	NO ACTION

A CASCADE DELETE szabály hatása: ha egy ProductRequest rekord törlésre kerül, automatikusan törlődnek a hozzá tartozó Chat rekordok is. Ha a Chat rekord törlődik, automatikusan törlődnek a ChatMessage rekordok is. Ez biztosítja az adatkonzisztenciát anélkül, hogy az alkalmazás kódjában külön törlési logikát kellene implementálni.

8.2 Adatbázis indexek és azok indoklása

Az adatbázisban az alábbi indexek kerültek létrehozásra a leggyakoribb lekérdezési minták optimalizálásához:

Index neve	Tábla	Indexelt mező(k)	Lekérdezési minta	Várható teljesítményjavulás
IX_Products_CategoryId	Products	ProductCategoryId	Kategória szerinti szűrés a terméklistán	Full table scan helyett index scan
IX_Products_SenderId	Products	SenderId	Adományozó saját termékeinek lekérése	Gyors lekérés azonosító alapján
IX_Products_CountyId	Products0	CountyId	Megye szerinti szűrés	Hatékony regionális szűrés
IX_ProductRequests_RequesterId	ProductRequests	RequesterId	"Saját igényléseim" lista betöltése	Gyors felhasználói szűrés
IX_ProductRequests_ProductId	ProductRequests	ProductId	Van-e aktív igénylés termékre?	Azonnali foglaltság-ellenőrzés
IX_ChatMessage_ChatId	ChatMessage	ChatId	Chat előzmények betöltése	Gyors üzenettörténet betöltés

8.3 Az adatbázis-séma teljes létrehozó szkriptjének részletes magyarázata

Az adatbázis-séma a ErtekmentRendszer áttekintés_db.sql fájlban található. A szkript végrehajtásának sorrendje kritikus a külső kulcsos kapcsolatok miatt. Az alábbiakban a táblák létrehozásának sorrendje és az egyes döntések részletes magyarázata olvasható:

- Counties és ProductCategory táblák: Ezek a táblák más tábláktól nem függenek, ezért elsőként kerülnek létrehozásra. A Counties tábla feltöltése statikus – a 20 vármegye soha nem változik. A ProductCategory IsActive mezője lehetővé teszi a kategóriák ideiglenes letiltását anélkül, hogy a létező termékekre hivatkozó rekordok sérülnének.
- Users tábla: A felhasználói tábla a második, mivel a Products és ProductRequests táblák hivatkoznak rá. A UserRole integer helyett enumként van kezelve a C# domain modellben, ami típusbiztos kódot eredményez. A UserStatus mező alapértelmezett értéke 1 (Active), így az adminisztrátornak csak negatív esetben kell beavatkoznia.
- Products tábla: Három külföldi kulcsot tartalmaz (Users, ProductCategory, Counties). Az ImagePath NVARCHAR(500) típusa elegendő a GUID-alapú fájlneveknek. A ProductStatus alapértelmezett értéke az alkalmazás kódjában kerül beállításra (Active), nem az adatbázis-szkriptben, hogy az EF Core migrációk megfelelően kezeljék.
- ProductRequests tábla: A CASCADE DELETE a Products tábla felé biztosítja, hogy ha egy termék tényleges törlése megtörténne (jelen implementációban soft delete alkalmazott), az igénylések is törlődnének. A ProcessedAt mező NULL értékű Pending állapotnál, és csak lezáráskor kerül feltöltésre.
- RequesterLimitRule és RequesterLimitUsage: A UNIQUE constraint az UC_RequesterLimitUsage (RequesterId, RuleId, PeriodStart) kombinációján biztosítja, hogy egy felhasználóhoz, egy szabályhoz és egy periódus-kezdethez kizárólag egy sor tartozzon, megakadályozva az adatduplikációt párhuzamos igénylések esetén.

- Chat és ChatMessage táblák: A Chat tábla UNIQUE constraint-je a ProductRequestId mezőn biztosítja, hogy minden igényléshez legfeljebb egy chat-szoba jöjjön létre. Az alkalmazáskód is ellenőrzi ezt, de az adatbázis-szintű constraint adja az utolsó védelmi vonalat.

9. Melléklet – A Blazor WEB réteg részletes bemutatása

Az Értékmentő frontend-je Blazor Server technológián alapul. A Blazor Server a .NET ökoszisztéma részeként lehetővé teszi, hogy a felhasználói felület logikáját C# kóddal valósítsuk meg, amelynek végrehajtása a szerveren történik, az állapot SignalR kapcsolaton keresztül szinkronizálódik a böngészővel.

9.1 Az alkalmazás oldalainak listája

Fájl neve	URL	Szerepkör-feltétel	Funkció
Home.razor	/	Nyilvános	Főoldal: héró szekció, küldetés, eredmények, GYIK, kapcsolat
Login.razor	/login	Nyilvános	Bejelentkezési form JWT token fogadással
Register.razor	/register	Nyilvános	Regisztrációs form
Products.razor	/products	Requester	Termékek böngészése szűrőkkel és lapozással
MyProducts.razor	/my-products	Sender	Saját felajánlott termékek kezelése, feltöltés
ProductCreate.razor	/product-create	Sender	Új termék létrehozó form képfeltöltéssel

MyRequests.razor	/my-requests	Requester	Saját igénylések listája és kezelése
Chats.razor	/chats	Mindkettő	Chat-szobák listája olvasatlan darabszámmal
Chat.razor	/chat/{requestId}	Mindkettő	Valós idejű chat felület SignalR kapcsolattal
AdminUsers.razor	/admin/users	Admin	Felhasználók kezelése
AdminProducts.razor	/admin/products	Admin	Termékek adminisztrátori kezelése
AdminProductRequests.razor	/admin/requests	Admin	Igénylések adminisztrátori kezelése
AdminRequesterLimitRule.razor	/admin/limitrules	Admin	Limitszabályok CRUD kezelése

9.2 Hitelesítési architektúra a Blazor kliensen

A Blazor kliensen a hitelesítés az alábbi komponensek együttműködésével valósul meg:

42. CustomAuthProvider: Az AuthenticationStateProvider-t felüldefiniáló osztály, amely a JWT token-t a böngésző LocalStorage-ából olvassa be. Ha a token érvényes és nem járt le, az azonosítás sikeresnek minősül, és a felhasználói adatok (azonosító, szerepkör, e-mail) a token claim-jeiből kerülnek kinyerésre.

43. TokenStore: Egy scoped service, amely az aktuális session JWT tokenjét tartja memóriában, elérhetővé téve minden API-hívó ApiService-osztály számára anélkül, hogy azoknak közvetlenül a LocalStorage-hoz kellene hozzáférniük.
44. LocalStorageService: JavaScript interop segítségével ír és olvas a böngésző localStorage-ába. A token a „authToken” kulcs alatt tárolódik.
45. AuthApiService, ProductApiService, stb.: Ezek az ApiService osztályok az HttpClient-et használják az API-kérések küldéséhez. Minden hitelesített kéréshez hozzáadják az Authorization: Bearer {token} fejléct a TokenStore-ból kiolvasott token alapján.

9.3 Az oldalszintű jogosultság-ellenőrzés

Minden szerepkör-korlátozott oldal az OnAfterRenderAsync életrajzi-metódusban végzi el a hitelesítés ellenőrzését. Az ellenőrzés csak az első render után történik, mivel a LocalStorage-hozzáférés csak kliens oldali renderelés után lehetséges. A logika:

```
await AuthServiceProvider.GetAuthenticationStateAsync();  
if (!authState.User.IsInRole("Requester"))  
    Navigation.NavigateTo("/login");
```

Ez a minta biztosítja, hogy a nem jogosult felhasználók automatikusan a bejelentkezési oldalra kerüljenek. Az Admin oldalak esetén az IsInRole("Admin") ellenőrzés kerül alkalmazásra.

9.4 A SignalR valós idejű chat implementáció

A chat valós idejű frissítéséhez a Blazor kliens HubConnection segítségével csatlakozik a szerver ChatHub-jához. A kapcsolat felépítésének lépései:

- A Chat.razor oldal betöltésekor HubConnectionBuilder segítségével kapcsolat jön létre a /chatHub URL-re, a JWT token Authorization fejlécként hozzáadva.
- A csatlakozás után a kliens meghívja a JoinChat(requestId.ToString()) metódust, amivel csatlakozik a megfelelő SignalR csoporthoz.

- A kliens regisztrál az ReceiveMessage szerver-eseményre. Amikor a szerver új üzenetet jelez, a kliensen automatikusan frissül az üzenetlista és a UI újra renderelődik.
- Az oldal elhagyásakor a kliens meghívja a LeaveChat metódust, majd a hub connection bezárul (IAsyncDisposable implementáción keresztül).

10. Melléklet – Konfigurációs fájlok és üzembe helyezés

10.1 Az appsettings.json konfiguráció

Az API réteg konfiguráció az appsettings.json fájlban és a fejlesztői appsettings.Development.json fájlban tárolódik. A production környezetben érzékeny értékeket (JWT titkos kulcs, kapcsolati karakterlánc) environment variable-ökből kell beolvasni.

Konfiguráció-kulcs	Értékleírás	Biztonsági szempontok
ConnectionStrings:DefaultConnection	Az SQL Server kapcsolati karakterlánc (szerver, adatbázis neve, hitelesítési mód)	Production-ben ne kerüljön a forráskódba. Secrets Manager vagy env variable ajánlott.
Jwt:Key	A JWT aláíráshoz használt szimmetrikus titkos kulcs	Minimum 32 karakter, véletlenszerű érték. Production-ben env variable-ként tárolandó.
Jwt:Issuer	A token kibocsátójának neve (ErtekmentoAPI)	Informális érték, egyediség biztosítása ajánlott.
Jwt:DurationInMinutes	Token érvényességi ideje percben (60)	Rövidebb élettartam biztonságosabb, de kényelmetlen. 60-120 perc elfogadható.

AllowedWebOrigin	A Blazor kliens URL-je (CORS whitelist)	Production-ben a valódi domainnévvel kell feltölteni.
------------------	---	---

10.2 Program.cs – A middleware pipeline konfigurációja

A Program.cs fájl az ASP.NET Core alkalmazás indulásakor végrehajtott konfigurációs kódot tartalmazza. A middleware-ek sorrendje kritikus fontosságú:

Sorrend	Middleware neve	Szerepe
1	AddDbContext (EF Core)	Az adatbázis-kapcsolat regisztrálása a DI konténerbe SQL Server konfigurációval
2	AddScoped (Service-ek)	Az összes service és repository interfész-implementáció párosítás regisztrálása
3	AddCors (BlazorPolicy)	CORS-szabály hozzáadása a Blazor kliens URL-jével (AllowedWebOrigin)
4	AddAuthentication (Bearer)	JWT Bearer hitelesítés konfigurálása a titkos kulccsal
5	AddSwaggerGen	Swagger dokumentáció generálása Bearer tokennel ellátott végpontok teszteléséhez
6	UseHttpsRedirection	HTTP kérések automatikus átirányítása HTTPS-re
7	UseCors (BlazorPolicy)	CORS-szabály alkalmazása – CORS-nak az Auth ELŐTT kell lennie!
8	UseStaticFiles	A wwwroot mappa statikus fájljainak (képek) kiszolgálása

9	UseAuthentication	JWT token érvényesítése és a User principal beállítása
10	UseAuthorization	Jogosultság-ellenőrzés a [Authorize] attribútumok alapján
11	MapControllers	Az összes kontroller végpont hozzárendelése az útvonalakhoz

10.3 Fejlesztői környezet felállítása (Getting Started)

Az alkalmazás helyi fejlesztői környezetben való futtatásához az alábbi lépések szükségesek:

- Telepítse a szükséges szoftvereket: .NET 8 SDK (<https://dotnet.microsoft.com/download>), SQL Server Express (<https://www.microsoft.com/sql-server>), Visual Studio 2022 Community.
- Klónozza le a repository-t: `git clone https://github.com/your-org/ErtekmentoProject.git`
- Nyissa meg a `ErtekmentoProject.slnx` megoldásfájlt Visual Studio 2022-ben.
- Szerkessze az `API/appsettings.json` fájlban a `DefaultConnection` kapcsolati karakterláncot a saját SQL Server példányára.
- Futtassa a `Ertekmento_db.sql` szkriptet SQL Server Management Studio-ban. Ez létrehozza az adatbázist, a táblákat, az indexeket és a kezdő adatokat.
- Indítsa el az API projektet (F5 vagy Ctrl+F5). A Swagger felület `https://localhost:{port}/swagger` címen érhető el.
- Szerkessze a WEB projekt `appsettings.json` fájlában az API URL-t, hogy a Blazor kliens megtalálja a backend-et.
- Indítsa el a WEB projektet. Az alkalmazás alapértelmezetten a `https://localhost:7189` címen érhető el.

11. Melléklet – Részletes tesztesetek

Ez a melléklet az alkalmazás részletes funkcionális, negatív és határeset-tesztjeit tartalmazza, kiterjesztve az alapidokumentáció tesztleírásait.

11.1 Regisztrációs tesztek

TC-REG-001 Sikeres regisztráció – Sender szerepkör	
Előfeltétel	Nincs bejelentkezett felhasználó, az e-mail és felhasználónév szabad.
Lépések	1. Navigáljunk /register oldalra. 2. Töltsük ki: UserName=teszt_ado, Email=ado@test.hu, Password=Teszt123, FirstName=Teszt, LastName=Adó, RoleId=1. 3. Kattintsunk Regisztráció.
Várt eredmény	200 OK válasz, átirányítás /login oldalra. Az adatbázisban megjelenik az új felhasználó Active státusszal.
Tényleges eredmény	PASS

TC-REG-002 Regisztráció duplikált felhasználónévvel	
Előfeltétel	A "don" felhasználónév már létezik az adatbázisban.
Lépések	Regisztrációs kísérlet UserName=don értékkel.
Várt eredmény	400 Bad Request. A hibaüzenet: "A regisztráció sikertelen".
Tényleges eredmény	PASS

TC-REG-003 Regisztráció 5 karakteres jelszóval	
Előfeltétel	Nincs előfeltétel.
Lépések	Regisztrációs kísérlet Password=Ab123 (5 karakter) értékkel.
Várt eredmény	Kliens oldali validáció: "A jelszónak legalább 6 karakter hosszúnak kell lennie." Kérés el sem jut a szerverig.
Tényleges eredmény	PASS

11.2 Bejelentkezési tesztek

TC-LOG-001 Sikeres bejelentkezés e-maillal	
Lépések	Bejelentkezés EmailOrUserName=userSender@teszt.com, Password=teszt123 adatokkal.
Várt eredmény	200 OK + JWT token. A token localStorage-ban tárolódik. Átirányítás /.
Tényleges eredmény	PASS

TC-LOG-002 Bejelentkezés tiltott fiókkal	
Előfeltétel	A felhasználó UserStatus=Banned értékre állítva.

Lépések	Normál bejelentkezési kísérlet.
Várt eredmény	401 Unauthorized, hibaüzenet: "User not active"
Tényleges eredmény	PASS

11.3 Termékkezelési tesztek

TC-PRD-001 Termék feltöltése képpel – Sender	
Előfeltétel	Bejelentkezve Sender szerepkörrel.
Lépések	1. /my-products oldal. 2. Új termék gomb. 3. Kitöltés: név=Kerti szék, leírás=jó állapotban, kategória=Bútor, megye=Budapest. 4. Kép feltöltés: kerti_szek.jpg (300 KB). 5. Mentés.
Várt eredmény	Termék megjelenik a saját termékeim listában, képpel együtt. Az adatbázisban ProductStatus=Active.
Tényleges eredmény	PASS

TC-PRD-002 Termék törlése (soft delete)	
Előfeltétel	Egy Active státuszú termék létezik az adományozónál.
Lépések	A saját termékeim listán a termék Törlés gombjára kattintva megerősítés után törlés.

Várt eredmény	A termék eltűnik az adományozó listájából. Az adatbázisban ProductStatus=Deleted (tényleges törlés nem történt). Az igénylők listájáról is eltűnik.
Tényleges eredmény	PASS

11.4 Igénylési tesztek

TC-REQ-001	Igénylés – Limitszabály nélküli kategória
Előfeltétel	Active termék az Egyéb kategóriában. Nincs limitszabály erre a kategóriára.
Lépések	Requester bejelentkezés után terméket megnyitni, Igénylés gombra kattintani.
Várt eredmény	ProductRequest rekord létrejön Pending státusszal. Termék státusza Active→Pending lesz.
Tényleges eredmény	PASS

TC-REQ-002	Igénylés – Monthly 2 db limit (első igénylés)
Előfeltétel	Bútor kategóriában Monthly MaxQuantity=2 limitszabály aktív. Az igénylőnek 0 felhasznált igénylése van ebben a hónapban.
Lépések	Bútor kategóriás terméket igényel.
Várt eredmény	Sikeres igénylés. RequesterLimitUsage rekord létrejön PeriodStart=hónap első napja, UsedQuantity=1 értékekkel.

Tényleges eredmény	PASS
--------------------	------

TC-REQ-003 Igénylés – Monthly 2 db limit elérve	
Előfeltétel	Bútor kategóriában a felhasználónak UsedQuantity=2 az aktuális hónapban.
Lépések	Harmadik bútor terméket próbál igényelni.
Várt eredmény	400 Bad Request. ClaimResultDto.Success=false, hibaüzenet: "Elérted a havi limitet ebben a kategóriában."
Tényleges eredmény	PASS

11.5 Chat tesztek

TC-CHT-001 Első üzenet küldése – chatszoba automatikus létrehozás	
Előfeltétel	Igénylő beadott igénylést. Chat tábla üres erre a requestId-re.
Lépések	Az igénylő a chat felületen üzenetet küld.
Várt eredmény	Új Chat rekord jön létre. Új ChatMessage rekord is létrejön. Az adományozó valós időben látja az üzenetet (SignalR).
Tényleges eredmény	PASS

TC-CHT-002 Olvasatlan üzenetek számlálója	
Előfeltétel	Az adományozónak van egy megnyílatlan üzenete egy Pending igénylés chatjában.
Lépések	Az adományozó bejelentkezik és megtekinti a fejléc chat-badge.
Várt eredmény	A badge 1-et mutat. Miután a chat megnyílik és MarkAsAllRead meghívódik, a badge 0-ra csökken.
Tényleges eredmény	PASS

11.6 Adminisztrátori tesztek

TC-ADM-001 Admin felhasználó tiltása	
Előfeltétel	Adminisztrátorként bejelentkezve. Egy Active felhasználó létezik.
Lépések	Admin → Felhasználók → Tiltás gomb a kiválasztott felhasználón.
Várt eredmény	UserStatus=Banned az adatbázisban. A tiltott felhasználó következő bejelentkezési kísérletkor 401 hibát kap.
Tényleges eredmény	PASS

TC-ADM-002 Limitszabály létrehozása és azonnali érvényesülése	
Előfeltétel	Nincs limitszabály az Elektronika kategóriához.
Lépések	1. Admin → Limitszabályok → Új szabály. 2. Kategória=Elektronika, PeriodType=Monthly, MaxQuantity=1. 3. Mentés.
Várt eredmény	Limitszabály megjelenik a listában. Következő igénylésnél a rendszer ellenőrzi az új szabályt.
Tényleges eredmény	PASS

11.7 Reszponzivitás és böngésző-kompatibilitás tesztek

Tesztkörnyezet	Böngésző	Felbontás	Eredmény	Megjegyzés
Asztali gép	Chrome 123	1920×1080	PASS	Teljes funkcionalitás rendben
Asztali gép	Firefox 124	1920×1080	PASS	Teljes funkcionalitás rendben
Asztali gép	Edge 123	1920×1080	PASS	Teljes funkcionalitás rendben
Laptop	Chrome 123	1366×768	PASS	Kisebb szűrő-blokk, de olvasható
Táblagép (Chrome DevTools)	iPad Pro nézet	1024×1366	PASS	Kártya elrendezés 2 oszlopos

Okostelefon Safari	iPhone 16 Pro	390×844	PASS	Egyoszlopos elrendezés, hamburger menü
Okostelefon (Chrome DevTools)	Galaxy S21	360×800	PASS	Kisebb kártyák, minden gomb elérhető

12. Melléklet – A projekt teljes könyvtárszerkezete

Az Értékmentő projekt Visual Studio Solution (.slnx) formátumban öt projektből áll. Az alábbiakban a teljes könyvtárszerkezet és a főbb fájlok célja kerül bemutatásra.

ErtekmentoProject/	
├─ API/	← REST API projekt
│ └─ Controllers/	
│ │ └─ AdminController.cs	← Admin CRUD
végpontok	
│ │ └─ AuthController.cs	← Regisztráció /
Bejelentkezés	
│ │ └─ ChatController.cs	← Chat API végpontok
│ │ └─ LimitController.cs	← Limit-ellenőrzés
és felhasználás	
│ │ └─ LookupController.cs	← Referenciaadat-
lekérdezések	
│ │ └─ ProductController.cs	← Termék CRUD
│ │ └─ ProductRequestController.cs	← Igénylések
kezelése	
│ └─ Program.cs	← Konfiguráció, DI
regisztrálás	
│ └─ appsettings.json	← Konfiguráció
│ └─ wwwroot/images/products/	← Feltöltött képek
tárhelye	
└─ Application/	← Üzleti logika réteg
└─ DTOs/	← Adatátviteli
objektumok	
│ │ └─ ChatInfoDto.cs	
│ │ └─ ChatMessageRequestDto.cs	
│ │ └─ ChatMessageResponseDto.cs	
│ │ └─ ClaimResultDto.cs	
│ │ └─ LimitResponseDto.cs	
│ │ └─ LoginDto.cs	
│ │ └─ LoginResponseDto.cs	
│ │ └─ ProductDto.cs	
│ │ └─ ProductRequestDto.cs	
│ │ └─ RegisterDto.cs	
│ │ └─ RequesterLimitRuleDto.cs	
│ │ └─ UserDto.cs	

└─ Interfaces/ Repository interfészek	← Service és
└─ Services/ implementációk	← Service
└─ Domain/	← Üzleti modell réteg
└─ Entities/	← Adatbázis entitások
└─ Chat.cs	
└─ ChatMessage.cs	
└─ County.cs	
└─ Product.cs	
└─ ProductCategory.cs	
└─ ProductRequest.cs	
└─ RequesterLimitRule.cs	
└─ RequesterLimitUsage.cs	
└─ User.cs	
└─ Enums/	
└─ ProductStatus.cs Completed, Deleted	← Active, Pending,
└─ RequestStatus.cs Failed	← Pending, Completed,
└─ UserRole.cs Admin	← Sender, Requester,
└─ UserStatus.cs Suspended, Banned	← Active, Deleted,
└─ Infrastructure/ réteg	← Infrastruktúra
└─ Data/ErtekmentoDbContext.cs DbContext	← EF Core
└─ Hubs/ChatHub.cs	← SignalR Hub
└─ Repositories/	
└─ ChatRepository.cs	
└─ GenericRepository.cs	
└─ JwtTokenGenerator.cs	
└─ UnitOfWork.cs	
└─ WEB/ frontend	← Blazor Server
└─ Components/	
└─ Layout/	← MainLayout, SideNav
└─ Pages/ (oldalak)	← Razor komponensek
└─ Shared/ komponensek	← Újrafelhasználható

13. Melléklet – Enum értékek és státuszgépek

Az Értékmentő alkalmazásban négy fő enum osztály definiálja az entitások állapotait. Ezeket az enum-okat a Domain réteg definiálja, és mind az adatbázis integer mezői, mind a C# üzleti logika ezekre hivatkozik.

13.1 ProductStatus – Termék állapotgép

Enum érték	Integer értéke	Magyar neve	Mikor kerül ebbe az állapotba?	Lehetséges átmenetek
Active	1	Aktív / Elérhető	Termék feltöltésekor (CreateProduct), vagy Failed igénylés esetén visszaállítás	Pending (igénylés beadásakor)
Pending	2	Folyamatban	Sikeres igénylés beadásakor (ClaimProduct)	Completed (sikeres átadás), Active (sikertelen/törölt igénylés)
Completed	3	Átadva	Az adományozó lezárja az igénylést success=true-val	Végállapot – tovább nem módosítható
Deleted	4	Törölve	Az adományozó törli a terméket (soft delete)	Végállapot – tovább nem módosítható

13.2 RequestStatus – Igénylés állapotgép

Enum érték	Integer értéke	Magyar neve	Leírás
Pending	1	Függőben	Az igénylés beadva, az adományozó még nem reagált
Completed	2	Teljesítve	Az adományozó lezárta success=true-val, az átadás megtörtént
Failed	3	Sikertelen	Az adományozó lezárta success=false-szal, az átadás meghiúsult

13.3 UserRole – Felhasználói szerepkörök

Enum érték	Integer értéke	Hozzáférési szint	Elérhető funkciók
Sender	1	Adományozó	Termékek feltöltése, szerkesztése, törlése; beérkezett igénylések kezelése; chat
Requester	2	Igénylő	Termékek böngészése; igénylés leadása és visszavonása; chat
Admin	3	Adminisztrátor	Teljes hozzáférés: felhasználókezelés, termékmoderálás, igényléskezelés, limitszabályok

13.4 UserStatus – Felhasználói státuszok

Enum érték	Integer értéke	Leírás	Bejelentkezés lehetséges?
Active	1	Aktív, normál működésű fiók	Igen
Deleted	2	Törölt fiók (soft delete)	Nem (401 Unauthorized)
Suspended	3	Ideiglenesen felfüggesztett fiók	Nem (401 Unauthorized)
Banned	4	Véglegesen tiltott fiók	Nem (401 Unauthorized)

14. Melléklet – Biztonsági szempontok és megoldások

A webalkalmazások biztonsága kiemelkedő fontosságú, különösen, ha érzékeny felhasználói adatokat kezelnek. Az Értékmentő fejlesztése során az alábbi biztonsági szempontokat vettük figyelembe és implementáltuk.

14.1 Jelszóbiztonság – BCrypt hash-elés

A felhasználói jelszavak soha nem kerülnek plaintext formában az adatbázisba. Minden jelszó `BCrypt.Net.BCrypt.HashPassword()` metódussal kerül hash-elésre, amely a következő tulajdonságokkal rendelkezik:

- 46. Só (salt): A BCrypt minden jelszóhoz automatikusan egyedi, véletlenszerű sót generál, amelyet a hash-ben tárol. Ez megakadályozza a rainbow table alapú támadásokat.
- 47. Munka faktor (work factor): Az alkalmazás az alapértelmezett 11-es munka faktort alkalmazza, ami $2^{11} = 2048$ iterációt jelent. Ez elegendő védelmet nyújt a brute-force támadások ellen.
- 48. Hash hossza: A BCrypt mindig 60 karakter hosszú hash-t generál, ezért a PasswordHash mező `NVARCHAR(255)` típussal van definiálva.
- 49. Ellenőrzés: A bejelentkezéskor `BCrypt.Net.BCrypt.Verify(plaintext, hash)` metódus hasonlítja össze a beírt jelszót a tárolt hash-sel, anélkül, hogy az eredeti jelszót vissza kellene fejteni.

14.2 JWT token biztonság

A JSON Web Token alapú hitelesítés az alábbi biztonsági konfigurációkkal rendelkezik:

- 50. Aláírási algoritmus: HMAC-SHA256 (HS256) szimmetrikus algoritmus, amelyhez minimum 32 karakteres titkos kulcs szükséges.

51. Token élettartama: 2 óra (120 perc). Lejárt token esetén a `ValidateLifetime=true` beállítás automatikusan 401 Unauthorized hibát ad.
52. HTTPS-only: A token kizárólag HTTPS kapcsolaton kerül elküldésre az `Authorization: Bearer` fejlécben, megakadályozva a man-in-the-middle támadásokat.
53. Issuer és Audience validáció: A jelenlegi implementációban le van tiltva (`ValidateIssuer=false`, `ValidateAudience=false`) a fejlesztési könnyedség érdekében. Production környezetben ajánlott bekapcsolni.

14.3 CORS konfiguráció

A Cross-Origin Resource Sharing (CORS) konfiguráció a BlazorPolicy névvel ellátott policy-n keresztül biztosítja, hogy az API csak a Blazor kliens URL-jéről érkező kéréseket fogadja el. Az `AllowedWebOrigin` konfigurációs érték alapján csak egy konkrét origin engedélyezett, amely alapértelmezetten `https://localhost:7189`.

Az `AllowAnyMethod()` és `AllowAnyHeader()` beállítások lehetővé teszik az összes HTTP metódus és fejléc használatát az engedélyezett origin-ről. Production-ben szorosabb konfigurációt ajánlott alkalmazni.

14.4 SQL Injection védelem – EF Core ORM

Az alkalmazás Entity Framework Core ORM-et használ az adatbázis-kommunikációhoz. Az EF Core automatikusan paraméteres lekérdezéseket generál, ezért SQL injection támadások az ORM-rétegen keresztül nem valósíthatók meg. Közvetlen SQL-szöveg (raw SQL) az alkalmazásban nem szerepel.